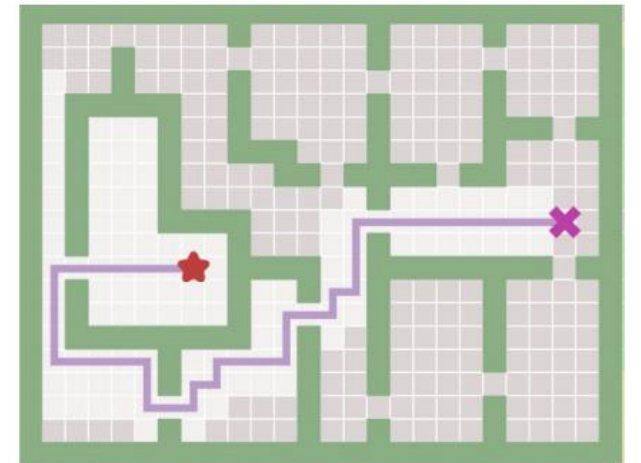


Module 5

PATH PLANNING

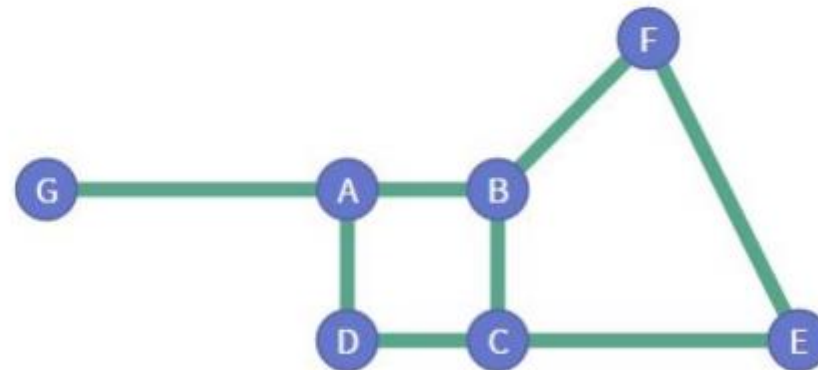
- Path planning in robotics is the **process of determining a sequence of movements** for a robot to navigate from its current location to a desired goal location while avoiding obstacles or other constraints in the environment.
- It involves finding the most efficient and safe path for the robot to follow.
- Path planning is crucial for enabling robots to autonomously navigate through complex environments.



Graph search

- Graph search path planning is a method used in robotics for finding a path from a starting point to a goal point in a given environment represented as a graph.
- In this approach, the environment is represented as a graph where nodes represent locations or states
- and edges represent possible transitions or connections between these locations.

Graphs have nodes and edges.

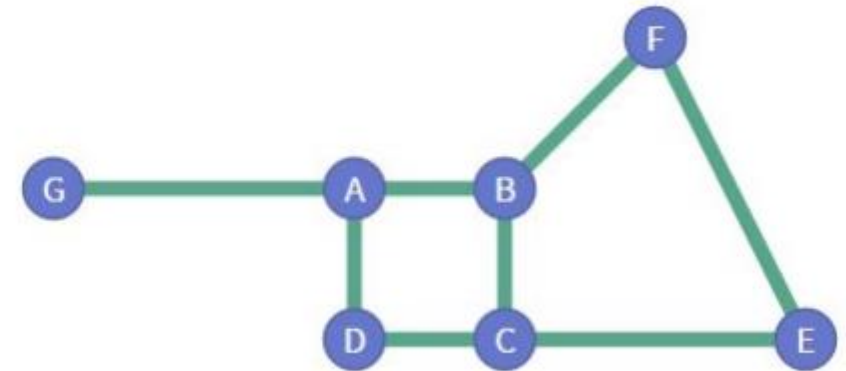


- Graph search path planning algorithms explore this graph to find a path that satisfies certain criteria, such as minimizing distance traveled or avoiding obstacles.
- These algorithms typically start at the initial node and systematically explore neighboring nodes, evaluating potential paths until the goal node is reached.

Graphs have **nodes** and **edges**.

- **Some common graph search path planning algorithms Include:**

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Dijkstra's Algorithm
- A*, D* Algorithm



4.2.3 Graph Traversal

There are 2 types of traversals:

1. Breadth First search
2. Depth First Search

BREADTH-FIRST SEARCH

The Breadth First Search (BFS) algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before moving on to the nodes at the next depth level. To avoid processing a node more than once, we divide the vertices into Visited and Not visited vertices. A Boolean visited array is used to mark the visited vertices. For simplicity, it is assumed that all vertices are reachable from the starting vertex.

Starting from the root, all the nodes at a particular level are visited first and then the nodes of the next level are traversed till all the nodes are visited.

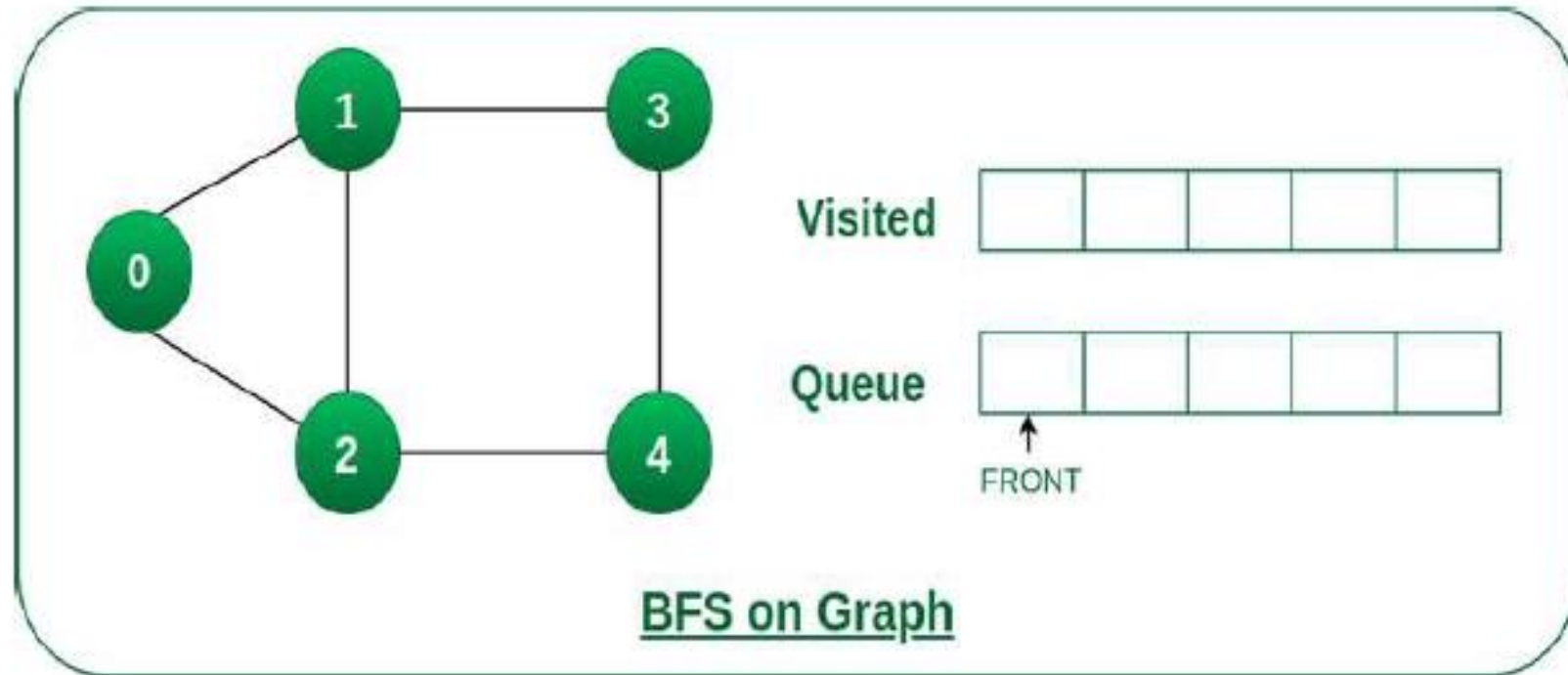
To do this a queue is used. All the adjacent unvisited nodes of the current level are pushed into the queue and the nodes of the current level are marked visited and popped from the queue.

Algorithm

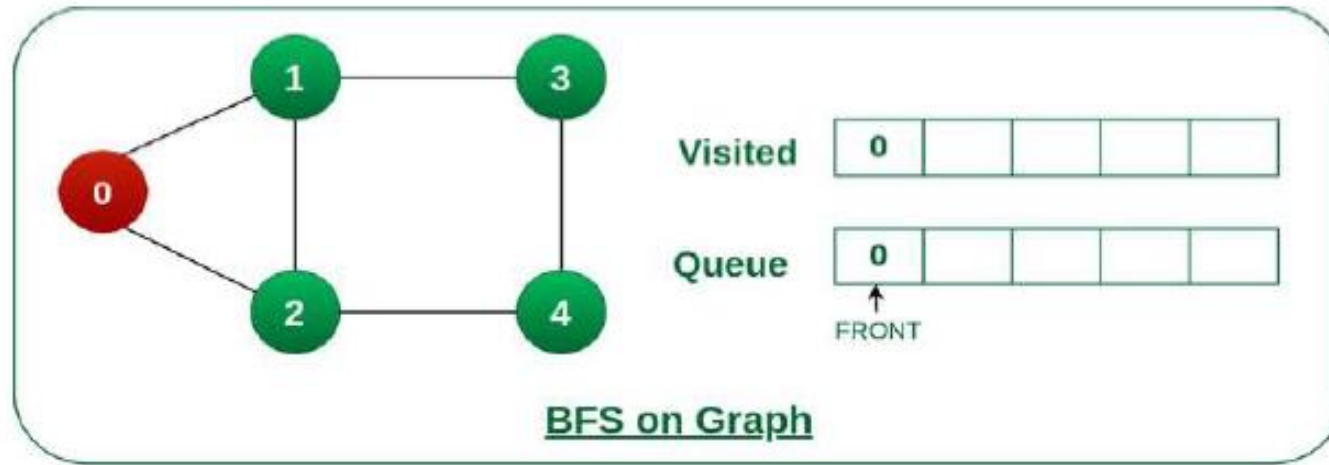
Steps

1. Initialize all the nodes as unvisited
2. Insert the first node/ starting node V_i to queue
3. Repeat 4 and 5 until queue is empty
4. Delete the node from Queue and mark it as visited
5. Insert the unvisited adjacent nodes of V_i to queue
6. Repeat until all the nodes are visited
7. Stop

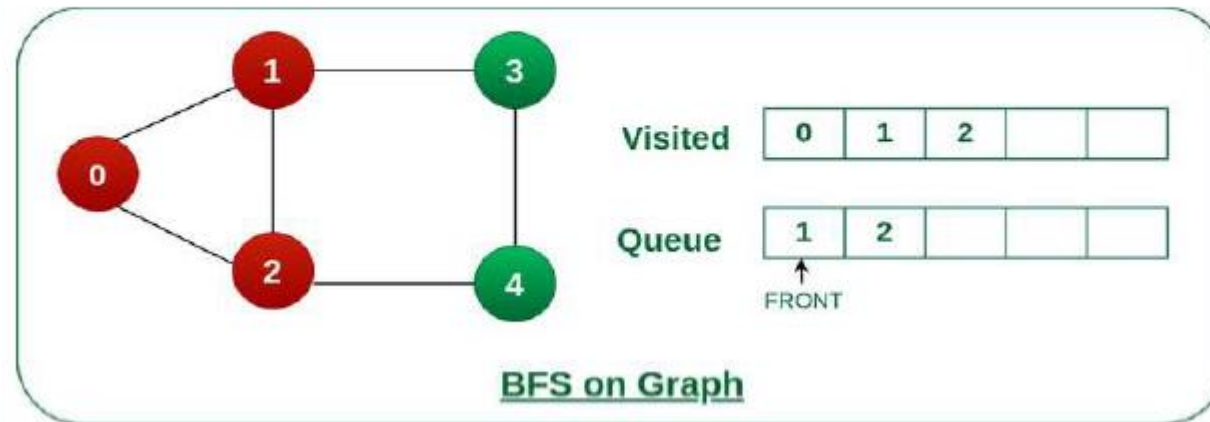
Step1: Initially queue and visited arrays are empty.



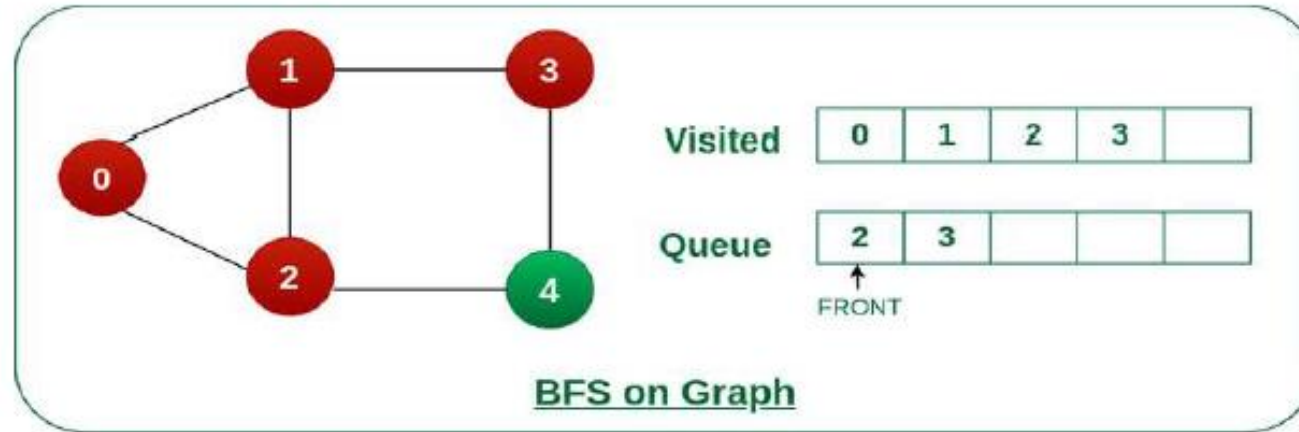
Step2: Push node 0 into queue and mark it visited.



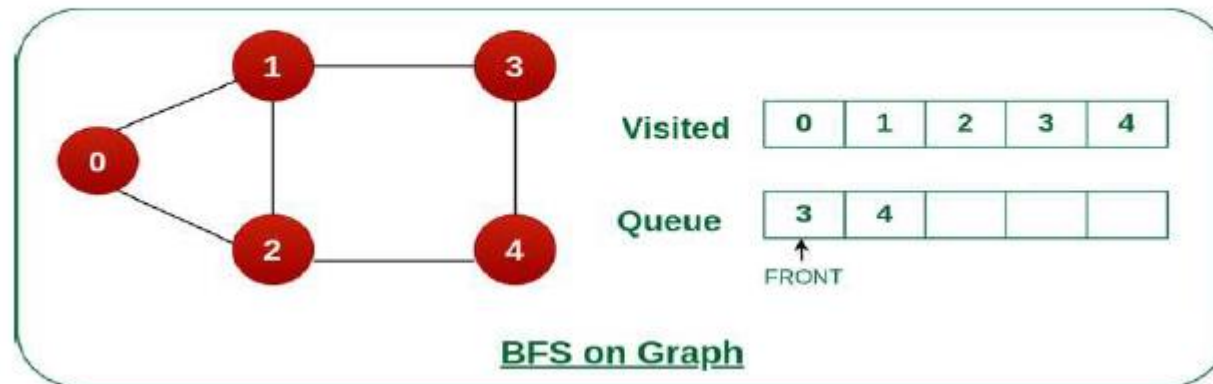
Step 3: Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.



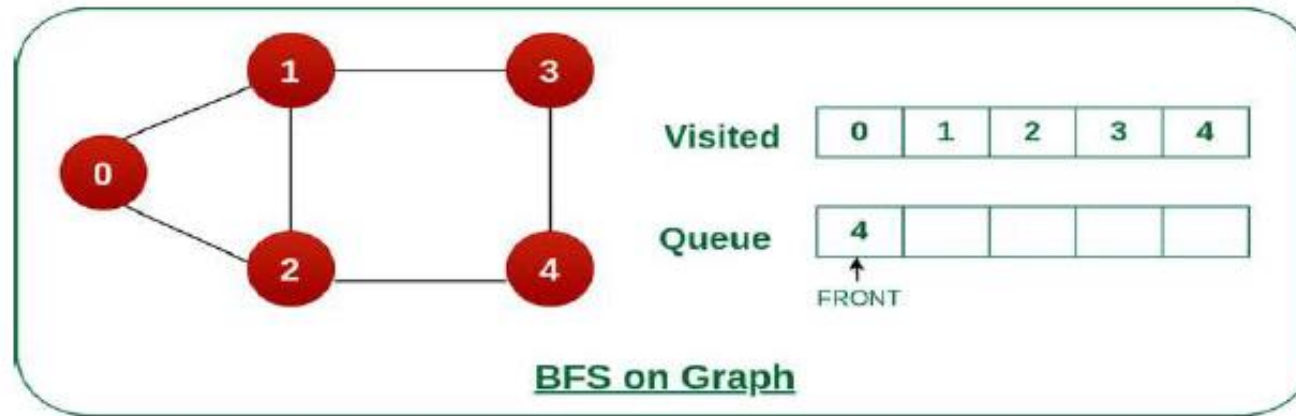
Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



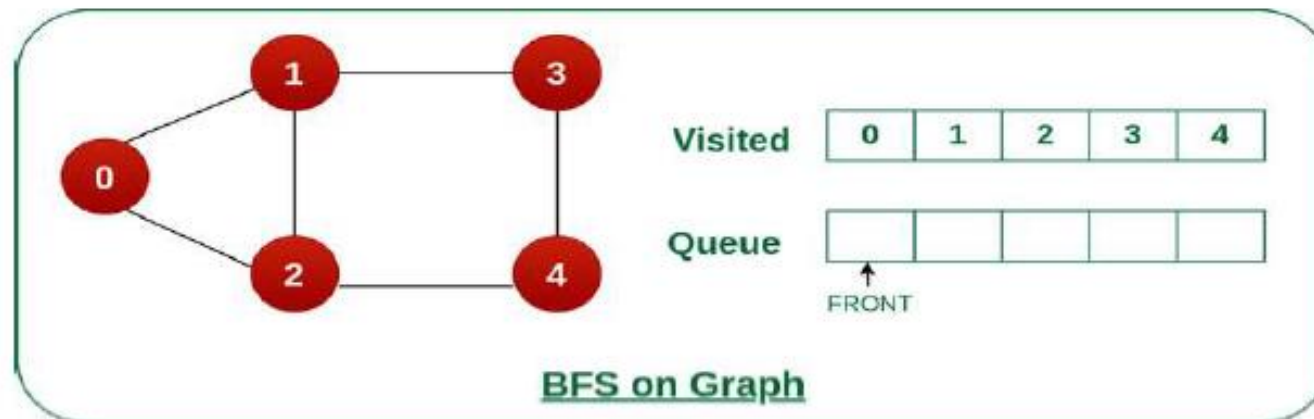
Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue. As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.



Steps 7: Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue. As we can see that every neighbours of node 4 are visited, and Queue becomes empty. So, terminate this process of iteration.



DEPTH-FIRST SEARCH

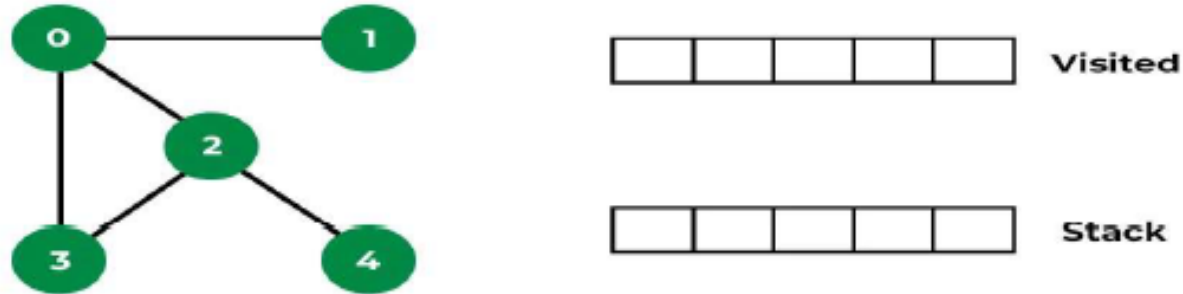
Algorithm

Steps

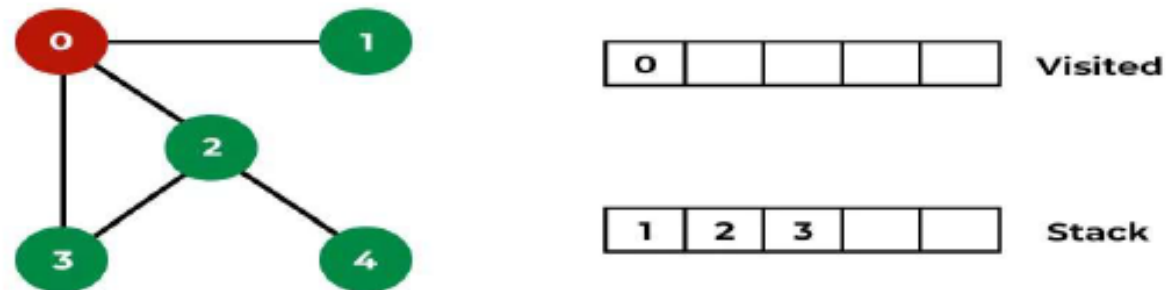
1. Initialize all the nodes as visited
2. PUSH the first node/ starting node V_i to stack
3. Repeat 4 and 5 until stack is empty
4. POP the top node of stack V_i and mark it as visited
5. PUSH the unvisited adjacent nodes of V_i to stack
6. Repeat until all the nodes are visited
7. Stop

Here the data structure used is STACK

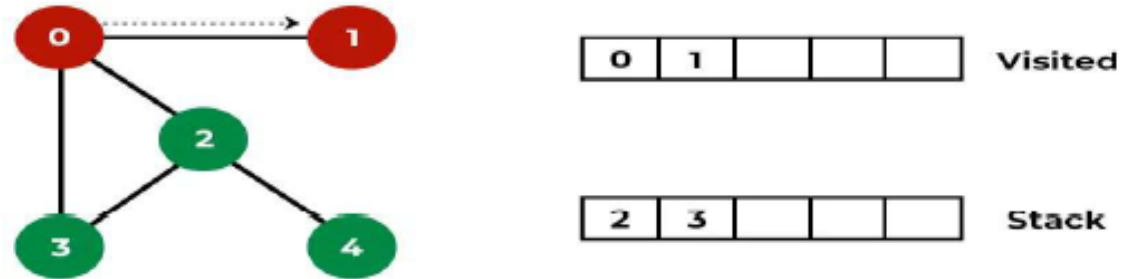
Step1: Initially stack and visited arrays are empty.



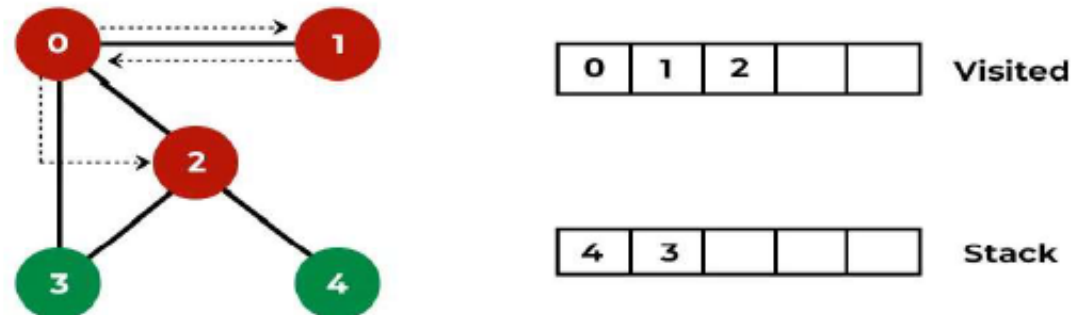
Step 2: Visit 0 and put its adjacent nodes which are not visited yet into the stack.



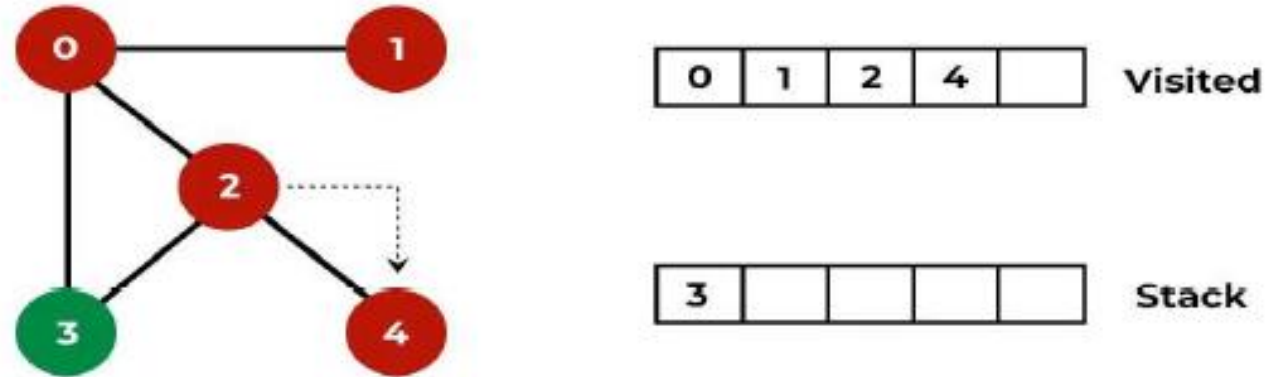
Step 3: Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



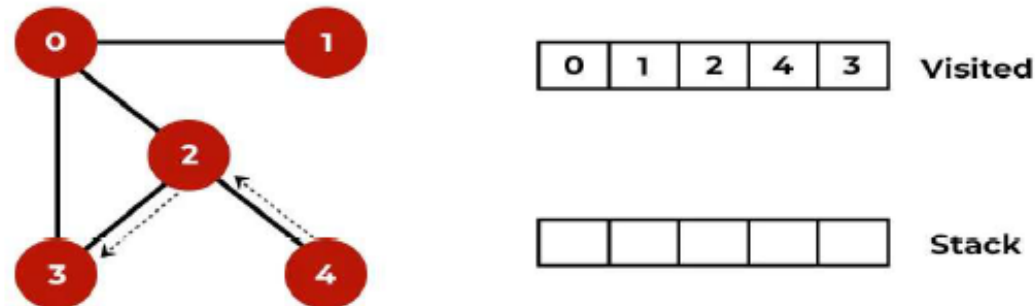
Step 4: Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.



Step 5: Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



Step 6: Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



III MODULE

There are many ways to design an algorithm for a given problem.

The following is a list of several popular design approaches.

- ⇒ Divide and Conquer
- ⇒ Greedy Approach
- ⇒ Dynamic Programming
- ⇒ Branch and Bound, and
- ⇒ Backtracking.

Divide and Conquer.

- ⇒ Divide the original problem into a set of sub problems.
- ⇒ Solve every sub problem individually recursively.
- ⇒ Combine the solutions of the sub problems into a solution of the whole original problem.

Greedy Approach

⇒ Greedy Algorithm seek to optimize a function by making choices locally.

⇒ The result is a good solution but not necessarily the best one.

⇒ It doesn't always guarantee the optimal solution, however generally produces solutions that are very close to the optimal solution.

Dynamic Programming

⇒ Generally used for optimization problems.

⇒ Techniques for efficiently computing recurrences by storing partial results.

⇒ It is a method of solving problems exhibiting the properties

of overlapping subproblems and optimal substructure.

Branch and Bound

⇒ It is a method for global optimization.

⇒ In a branch and bound algorithm, a given subproblem, which cannot be bounded has to be divided into at least two new restricted subproblems.

Backtracking Algorithms.

⇒ Backtracking algorithms try each possibility until they find the right one.

⇒ During the search, if an alternative does not work, the search backtracks to the choice point, and tries the next alternative.

⇒ If there are no more choice points, the search fails.

Divide and Conquer

Divide and Conquer is a technique for designing algorithms.

It consists of decomposing the instance to be solved into a number of smaller subinstances of the same problem, solving successively and independently each of these subinstances and combining the subsolutions thus obtained to obtain the solution of the original instance.

Divide and Conquer paradigm consists of following major steps

Divide

Divide the problem into several subproblems that are similar to the original problem but smaller

in size.

Conquer

Solve the subproblems recursively.

Combine

Combine the solutions of subproblems to create a solution to the original problem.

So divide and conquer strategy solves a problem by

⇒ Breaking it into subproblems that are themselves smaller instances of the same type of problem.

⇒ Recursively solving these subproblems.

⇒ Appropriately combining these answers.

Strassen's Matrix Multiplication

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} * \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Let $\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$ is the

resultant matrix produced after multiplication between two matrices

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

The resultant matrix is produced in classical matrix multiplication as follows:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

$$c_{22} = a_{21} b_{12} + a_{22} b_{22}$$

Using Strassen's matrix multiplication, the resultant matrix is obtained as follows:-

$$P_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$P_2 = (a_{21} + a_{22}) b_{11}$$

$$P_3 = (b_{12} - b_{22}) a_{11}$$

$$P_4 = (b_{21} - b_{11}) a_{22}$$

$$P_5 = (a_{11} + a_{12}) b_{22}$$

$$P_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$P_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

$$c_{11} = P_1 + P_4 - P_5 + P_7$$

$$c_{12} = P_3 + P_5$$

$$c_{21} = P_2 + P_4$$

$$c_{22} = P_1 + P_3 - P_2 + P_6$$

Comparison b/w classical and Strassen's matrix multiplication

→ In classical matrix multiplication it takes 8 multiplication to produce

the resultant matrix

So the recurrence equation

is

$$T(n) = 8T(n/2) + n^2$$

$$a = 8 \quad b = 2 \quad f(n) = n^2$$

$$n \log_b a = n \log_2 8 = n^3$$

$$\underline{f(n) < n \log_b a}$$

$$\begin{aligned} f(n) &= O(n \log_b a - \epsilon) \\ &= O(n^2) \end{aligned}$$

$$\begin{aligned} T(n) &= O(n \log_b a) \\ &= \underline{O(n^3)} \end{aligned}$$

But in Strassen's matrix multiplication, it takes only 7 multiplications to produce the result.

So the recurrence equation is

$$T(n) = 7T(n/2) + n^2$$

$$a = 7 \quad b = 2 \quad f(n) = n^2$$

$$n \log^3 n = n \log^3 7 = n^{2.80}$$

$$\underline{f(n) < n \log^3 n}$$

$$\begin{aligned} f(n) &= O(n \log^3 n - \epsilon) \\ &= O(n^2) \end{aligned}$$

$$\begin{aligned} T(n) &= O(n \log^3 n) \\ &= O(n^{2.81}) \end{aligned}$$

So we can conclude that the Strassen's matrix multiplication outperforms the classical matrix multiplication.

Example

Verify Strassen's matrix multiplication algorithm for the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

Ans

$$\begin{aligned} P_1 &= (a_{11} + a_{22})(b_{11} + b_{22}) \\ &= (1 + 1)(2 + 5) \\ &= 14 \end{aligned}$$

$$P_2 = (a_{21} + a_{22}) b_{11}$$

$$= (1+1) 2$$

$$= 4$$

$$P_3 = (b_{12} - b_{22}) a_{11}$$

$$= (3-5) 1$$

$$= -2$$

$$P_4 = (b_{21} - b_{22}) a_{22}$$

$$= (4-2) 1$$

$$= 2$$

$$P_5 = (a_{11} + a_{12}) b_{22}$$

$$= (1+1) 5$$

$$= 10$$

$$P_6 = (a_{21} - a_{11}) (b_{11} + b_{12})$$

$$= (1-1) (2+3)$$

$$= 0$$

$$P_7 = (a_{12} - a_{22}) (b_{21} + b_{22})$$

$$= (1-1) (4+5)$$

$$= 0$$

$$C_{11} = P_1 + P_4 - P_5 + P_7$$

$$= 14 + 2 - 10 + 0$$

$$= 6$$

$$C_{12} = P_3 + P_5$$

$$= -2 + 10$$

$$= 8$$

$$C_{21} = P_2 + P_4$$

$$= 4 + 2$$

$$= 6$$

$$C_{22} = P_1 + P_3 - P_2 + P_6$$

$$= 14 - 2 - 4 + 0$$

$$= 8$$

$$\begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$

So

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$

MERGE SORT

This algorithm was invented by John Von Neumann in 1945.

It follows the divide and conquer paradigm.

It works as follows:-

Divide

Divide n elements to be sorted into two subsequences of $n/2$ elements.

Conquer

Sort the two subsequences recursively.

Combine

Merge the two sorted subsequences to produce the sorted answer.

Algorithm

MergeSort(A, p, r)

if $p < r$

$q \leftarrow \lfloor (p+r)/2 \rfloor$

MergeSort(A, p, q)

MergeSort($A, q+1, r$)

Merge(A, p, q, r).

Merge(A, p, q, r)

$n_1 \leftarrow q - p + 1$

$n_2 \leftarrow r - q$

create arrays $L[1..n_1+1]$ and $R[1..n_2+1]$

for $i \leftarrow 1$ to n_1

$L[i] \leftarrow A[p+i-1]$

for $j \leftarrow 1$ to n_2

$R[j] \leftarrow A[q+j]$

$L[n_1+1] \leftarrow \infty$

$R[n_2+1] \leftarrow \infty$

$i \leftarrow 1$

$j \leftarrow 1$

for $k \leftarrow p$ to r

if $L[i] \leq R[j]$

$A[k] \leftarrow L[i]$

$i \leftarrow i + 1$

else

$A[k] \leftarrow R[j]$

$j \leftarrow j + 1$

Performance Analysis

Divide

The divide step just computes the middle of the subarray, which takes constant time $O(1)$.

Conquer

We recursively solve the subproblems, each of size $n/2$ which contributes $2T(n/2)$ to the running time.

Combine

Merge procedure on an n -element subarray takes $O(n)$ time.

$$\therefore f(n) = O(n \log n)$$

For best case, average case and worst case merge sort have the recurrence equation

$$T(n) = \begin{cases} O(1) & \text{if } n=1 \\ 2T(n/2) + n & \text{if } n>1 \end{cases}$$

$$T(n) = 2T(n/2) + n.$$

$$a=2 \quad b=2 \quad f(n)=n.$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$f(n) = n^{\log_b a}$$

$$f(n) = O(n^{\log_b a})$$

$$= O(n)$$

$$T(n) = O(n^{\log_b a} \log n)$$

$$= \underline{\underline{O(n \log n)}}$$

Merge Sort - Simple Example.

Merge Sort (15, 10, 5, 20) =

$$p=1, r=4 \therefore MS(A, p, q) \Rightarrow MS(A, 1, 4)$$

$$MS(A, 1, 4) \\ \{15, 10, 5, 20\}$$

$$\Rightarrow p < r \\ q = \left\lceil \frac{p+r}{2} \right\rceil \\ = \left\lceil \frac{1+4}{2} \right\rceil \\ = 2,$$

$$MS(A, p, q) \\ MS(A, 1, 2) \\ \{15, 10\}$$

$$< 2 \\ \Leftarrow \\ q = \left\lceil \frac{1+2}{2} \right\rceil \\ = 1$$

$$MS(A, p, q) \\ MS(A, 1, 1) \\ \{15\}$$

$p \neq r$

$$MS(A, q+1, r) \\ MS(A, 2, 2) \\ \{10\}$$

$p \neq r$

$$Merge(A, p, q, r) \\ \Rightarrow Merge(A, 1, 1, 2) \\ \{10, 15\}$$

$$MS(A, q+1, r) \\ MS(A, 3, 4) \\ \{5, 20\}$$

$$3 < 4 \\ \Leftarrow \\ q = \left\lceil \frac{3+4}{2} \right\rceil \\ = 3.$$

$$MS(A, p, q) \\ MS(A, 3, 3) \\ \{5\}$$

$p \neq r$

$$MS(A, q+1, r) \\ MS(A, 4, 4) \\ \{20\}$$

$p \neq r$

$$Merge(A, p, q, r) \\ Merge(A, 3, 3, 4) \\ \{5, 20\}$$

$$Merge(A, 1, 2, 4) \\ \{5, 10, 15, 20\}$$

$$\Rightarrow \underline{\underline{\{5, 10, 15, 20\}}}$$

GREEDY STRATEGY

Greedy Algorithms are simple and straight forward.

Greedy algorithms solve problems by making choice that seems best at the particular moment.

Many optimization problems that have no efficient solution can be solved using greedy algorithms.

Greedy algorithms may provide a solution that is close to optimal for these kinds of problems.

A greedy algorithm works if a problem exhibits the following two properties.

- 1- Greedy choice Property
- 2- Optimal Substructure.

Greedy choice Property

A Globally optimal solution can be arrived by making a locally optimal solution.

Optimal Substructure

Optimal solutions contain optimal sub solutions.

The greedy method suggests that one can devise an algorithm that works in stages, considering one input at a time.

At each stage, a decision is made regarding whether a particular input is or is not an optimal solution.

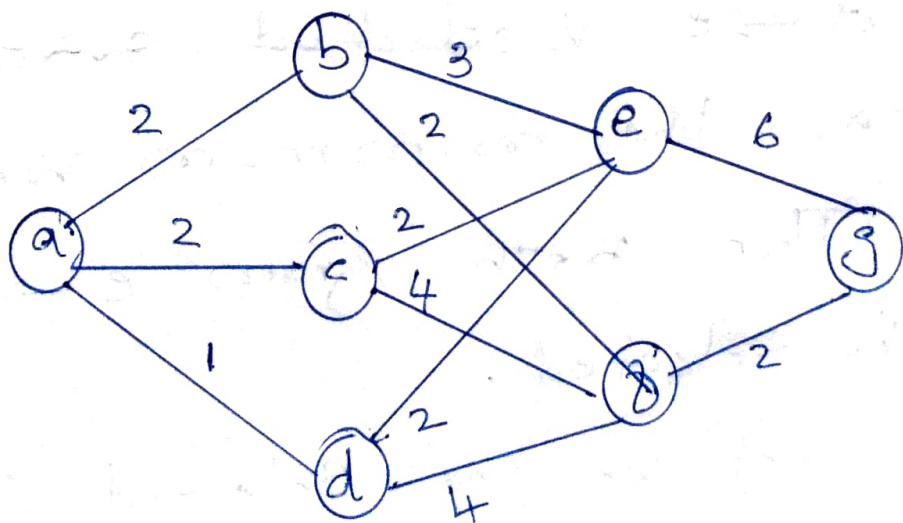
This is done by considering the inputs in an order determined by some selection procedure.

If the inclusion of the next input into the partially constructed optimal solution will result in an infeasible solution, then this input is not added to the partial solution.

Otherwise it is added.

Example

We want to find out the shortest path from a to g in the following graph using greedy strategy.



Let $SC(x)$ be the minimum distance required to reach 'g' from a node 'x'.

Greedy Algorithm will work in stages.

At the first stage it checks path from a.

[i.e., $a \rightarrow b$, $a \rightarrow c$, $a \rightarrow d$]

Path with minimum weight will be selected first

i.e., $a \rightarrow d$ is selected.

Then in the next stage path from d is checked.

[i.e. $d \rightarrow e$, $d \rightarrow g$]

$d \rightarrow e$ is selected, since it has the minimum weight.

Then path from e $\rightarrow$ g is selected.

Here the selected path is $a \rightarrow d \rightarrow e \rightarrow g$. and the weight is

$$1 + 2 + 6 \Rightarrow 9$$

But it is not an optimal solution.

Here we get only near optimal solution.

Control Abstraction for Greedy Method

Greedy(a, n)

$soln = \phi$

for $i = 0$ to n

$s = \text{select}(a);$

if feasible($soln, s$)

$soln = \text{Union}(soln, s)$

else

reject(s)

Return $soln$.

Here a selection of solution from the given input domain is performed using $s = \text{select}(a)$.

Then feasibility of the solution is checked using feasible (solution, S).

If it is feasible, it is added to the solution using union operation $\text{union}(\text{solution}, S)$.

Otherwise it is rejected.

Fractional Knapsack Problem.

Given n objects with weight w_i and value v_i and a knapsack which can hold at most weight W .

The objective is to obtain the filling of knapsack, that maximize the total profit.

Since the knapsack has capacity W , total weight of all the chosen items must be at most W .

Mathematically, we can represent

$$\begin{aligned} \text{it } \infty \quad & \text{Max } \sum_{1 \leq i \leq n} x_i V_i \\ \text{Sub. to } & \sum_{1 \leq i \leq n} w_i x_i \leq W, \text{ where} \\ & 0 \leq x_i \leq 1 \text{ and } 1 \leq i \leq n. \end{aligned}$$

Knapsack problem can be of two types.

$\Rightarrow$ 0/1 knapsack problem, and

$\Rightarrow$ Fractional knapsack problem.

0/1 Knapsack Problem

Here each item to be packed must take as whole or not.

i.e., items may not be broken into smaller pieces

Fractional Knapsack Problem

Here fractions of items can be taken rather than having to make a binary [0-1] choice for each item.

Algorithm

Fractional-knapsack (n, v, W)

for $i \leftarrow 1$ to n

$x[i] \leftarrow 0$

Weight $\leftarrow 0$

While Weight $< W$.

$i \leftarrow$ best remaining object

If Weight + $w[i] \leq W$

$x[i] \leftarrow 1$

Weight $\leftarrow$ Weight + $w[i]$

else

$x[i] \leftarrow \frac{W - \text{Weight}}{w[i]}$

Weight $\leftarrow$ Weight + $[x[i] * w[i]]$

Here the best remaining object can be selected three ways. They are

⇒ Lightest Weight

Object with least weight is selected first.

⇒ Most Valuable

Object with high value is selected first.

Value / Weight ratio

Object with high value / weight value is selected first.

Example

Consider the following instance of knapsack problem, $n=5$, $W=100$

$$(W_1, W_2, \dots, W_5) = (10, 20, 30, 40, 50)$$

$$(V_1, V_2, \dots, V_5) = (20, 30, 66, 40, 60)$$

Solution

Lightest Weight

for $i \leftarrow 1$ to n
 $i \leftarrow 1$ to 5

$$x[i] \leftarrow 0$$

$$x[i] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{weight} \leftarrow 0$$

$$\text{while } \text{weight} < W$$

$$\text{weight} < 100$$

$i \leftarrow$ best remaining object

Here object with least weight is selected as best remaining object.

$i \leftarrow 1$

if $\text{Weight} + w[i] \leq W$

$$0 + 10 < 100 \Rightarrow \text{True}$$

$x[i] \leftarrow 1$

$$\text{Weight} \leftarrow 0 + 10 = 10$$

$10 < 100 \Rightarrow \text{True}$

$i \leftarrow 2$

if $10 + 20 \leq 100 \Rightarrow \text{Yes}$

$x[2] \leftarrow 1$

$$\text{Weight} \leftarrow 10 + 20 = 30$$

$30 < 100 \Rightarrow \text{True}$

$i \leftarrow 3$

if $30 + 30 \leq 100 \Rightarrow \text{Yes}$

$x[3] \leftarrow 1$

$$\text{Weight} \leftarrow 30 + 30 = 60$$

$$x[i] \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$x[i] \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$x[i] \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\underline{60 < 100} \Rightarrow \text{True}$$

$$i \leftarrow 4$$

$$y \quad 60 + 40 \leq 100 \Rightarrow \text{Yes}$$

$$a[4] \leftarrow 1$$

$$\text{Weight} = 60 + 40 \Rightarrow 100$$

$$\underline{100 < 120} \Rightarrow \text{No}$$

exit

$a[1]$	1
$[2]$	1
3	1
4	1
25	0

Selected objects are 1, 2, 3 and
4th object

$$\text{Total Value} = 20 + 30 + 66 + 40$$
$$\Rightarrow 156$$

Most Valuable

Here object with highest value
is selected as the best remaining
object.

for $i \leftarrow 1$ to 5

$x[i] \leftarrow 0$

$x[1] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

Weight $\leftarrow 0$

$0 < 100 \Rightarrow \text{True}$

$i \leftarrow 3$

if $30 + 0 \leq 100 \Rightarrow \text{Yes}$

$x[3] \leftarrow 1$

Weight $\leftarrow 30 + 0 \Rightarrow 30$

$30 < 100 \Rightarrow \text{True}$

$i \leftarrow 5$

if $30 + 50 \leq 100 \Rightarrow \text{Yes}$

$x[5] \leftarrow 1$

Weight $\leftarrow 30 + 50 = 80$

$80 < 100 \Rightarrow \text{True}$

$i \leftarrow 4$

if $80 + 40 \leq 100 \Rightarrow \text{No}$
goto else case.

$x[1] \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

$x[1] \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$

$$x[i] = W - \text{Weight} / w[i]$$

$$= \frac{100 - 80}{40} = \frac{20}{40} = \frac{1}{2}$$

$$\begin{aligned} \text{Weight} &= \text{Weight} + [x[i] * w[i]] \\ &= 80 + \frac{1}{2} * 40 \\ &= 100 \end{aligned}$$

$$x[i] \begin{bmatrix} 0 \\ 0 \\ 1 \\ \frac{1}{2} \\ 1 \end{bmatrix}$$

$$100 < 100 \Rightarrow \text{No}$$

exit

Selected objects are 3rd, 5th and $\frac{1}{2}$ portion of 4th item.

Value / weight ratio method

Here object with high value / weight ratio is selected as best remaining object.

V	20	30	66	40	60
W	10	20	30	40	50
V/W	2	1.5	2.2	1	1.2

for $i \leftarrow 1$ to 5

$x[i] \leftarrow 0$

$x[i] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

Weight $\leftarrow 0$

$0 < 100 \Rightarrow \text{True}$

$i \leftarrow 3$

$0 + 30 \leq 100 \Rightarrow \text{Yes}$

$x[3] \leftarrow 1$

Weight $\leftarrow 0 + 30 = 30$

$30 < 100 \Rightarrow \text{Yes}$

$i \leftarrow 1$

$30 + 10 \leq 100 \Rightarrow \text{Yes}$

$x[1] = 1$

$x[i] \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

$x[i] \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

$$\text{Weight} = 30 + 10 = 40$$

$$\underline{40 < 100} \Rightarrow \text{Yes}$$

$$i \leftarrow 2$$

$$\text{if } 40 + 20 \leq 100 \Rightarrow \text{Yes}$$

$$x[2] = 1$$

$$\text{Weight} = 40 + 20 = 60$$

$$\underline{60 < 100} \Rightarrow \text{Yes}$$

$$i \leftarrow 5$$

$$\text{if } 60 + 50 \leq 100 \Rightarrow \text{No}$$

go to else case

$$x[5] = \frac{100 - 60}{50} = \frac{4}{5}$$

$$\text{Weight} = 60 + \left[\frac{4}{5} * 50 \right]$$

$$= 60 + 40$$

$$= 100$$

$$\underline{100 < 100} \Rightarrow \text{No}$$

exit

$$x[i] \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ \frac{4}{5} \end{bmatrix}$$

Selected objects
all 1st, 2nd, 3rd &
 $\frac{4}{5}$ th portion of
5th object with
profit 164.

Q. Find the optimal solution to a knapsack

with capacity $n=8$, $m=110$,

$(P_1, P_2 \dots P_8) = (11, 21, 31, 33, 43, 53, 55, 65)$

and

$(W_1, W_2 \dots W_8) = (1, 11, 21, 33, 43, 53, 55, 65)$

Ans:

	1	2	3	4	5	6	7	8
P	11	21	31	33	43	53	55	65
W	1	11	21	33	43	53	55	65
P/W	11	1.9	1.47	1	1	1	1	1
Initial $x[i]$	0	0	0	0	0	0	0	0
$x[i]$	1	1	1	1	1	$\frac{1}{53}$		

$$W_t \leftarrow 0$$

$$* \quad 0 < 110 \Rightarrow \text{True}$$

$$i \leftarrow 1$$

$$0 + 1 \leq 110 \Rightarrow \text{True}$$

$$x[1] \leftarrow 1$$

$$W_t \leftarrow 1$$

$$* \quad 1 < 110 \rightarrow \text{True}$$

$$i \leftarrow 2$$

$$1 + 11 \leq 110 \rightarrow \text{True.}$$

$$a[2] \leftarrow 1$$

$$wt \leftarrow 1 + 11 \Rightarrow 12$$

$$* \quad 12 < 110 \rightarrow \text{True.}$$

$$i \leftarrow 3$$

$$12 + 21 \leq 110 \rightarrow \text{True.}$$

$$a[3] \leftarrow 1$$

$$wt = 12 + 21 = 33$$

$$* \quad 33 < 110 \rightarrow \text{True.}$$

$$i \leftarrow 4$$

$$33 + 33 \leq 110 \rightarrow \text{True.}$$

$$a[4] \leftarrow 1$$

$$wt \leftarrow 66.$$

$$* \quad 66 < 110 \rightarrow \text{True.}$$

$$i \leftarrow 5$$

$$66 + 43 \leq 110 \rightarrow \text{True.}$$

$$a[5] \leftarrow 1$$

$$wt \leftarrow 109.$$

$$* \quad 109 < 110 \rightarrow \text{True}$$

$$i \leftarrow 6$$

$$109 + 53 \leq 110 \rightarrow \text{False}$$

$$x[i] \leftarrow \frac{110 - 109}{53} = \frac{1}{53}$$

$$\text{hit} \leftarrow 109 + \frac{1}{53} * 53$$

$$\leftarrow 109 + 1 \Rightarrow 110$$

$$110 < 110 \rightarrow \text{False}$$

$\Rightarrow$ Exit.

$\Rightarrow$ Selected objects are 1st, 2nd, 3rd, 4th, 5th and $\frac{1}{53}$ th fraction of 6th object.

Minimum Spanning Tree

A spanning tree T , for a graph $G: (V, E)$ is a subgraph $G' = (V', E')$ such that the following conditions hold.

1 $\Rightarrow$ The vertex set V' is same as that of graph G .

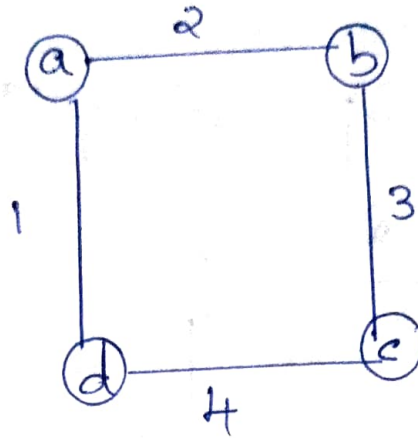
2 $\Rightarrow$ The edge set E' is a subset of E .

3 $\Rightarrow$ And there is no cycle.

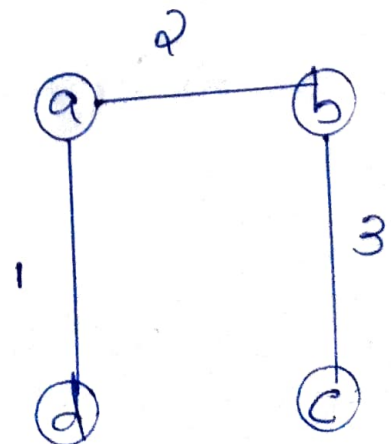
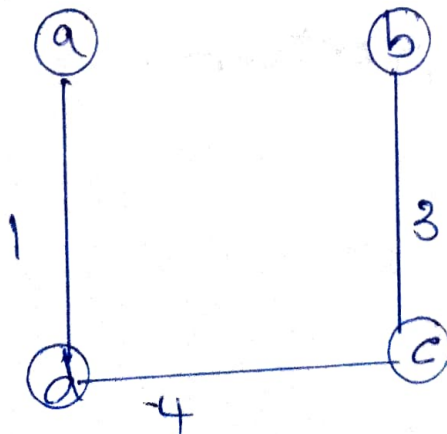
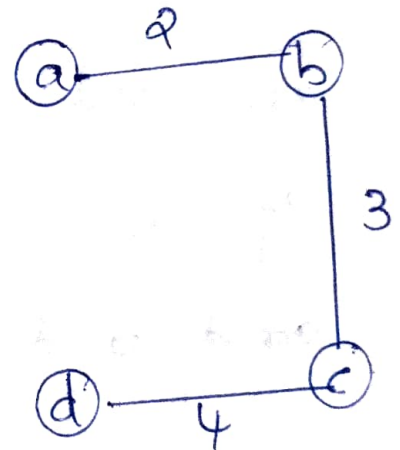
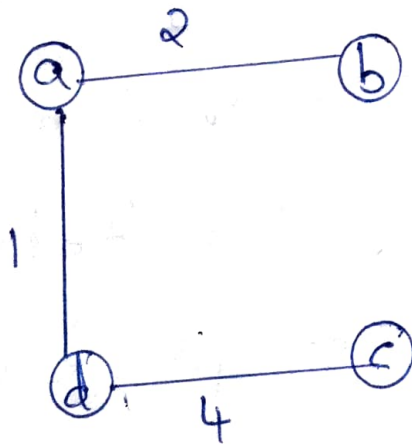
A minimum spanning tree [MST] for a weighted graph is a spanning tree with minimum weight.

For example :- Let $G = (V, E)$ is

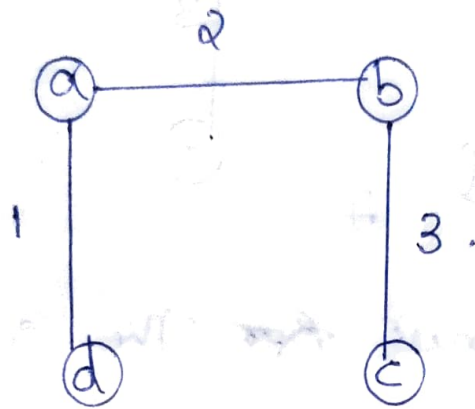
given



Spanning trees for the above graph
are shown below



In this, spanning tree with minimum weight is known as Minimum Spanning Tree.



Two different algorithms that are used to obtain minimum spanning tree of connected weighted and undirected graph are

⇒ Kruskal's algorithm, and

⇒ Prim's Algorithm.

Kruskal's Algorithm

This is one of the popular algorithms developed by Joseph Kruskal.

To create a minimum spanning tree using Kruskal's, we begin by choosing the edge with the minimum cost.

Algorithm works as follows:-

Suppose $G = (V, E)$ is a graph and T is a minimum spanning tree of graph G .

- 1: Initialize the spanning tree T to contain all the vertices in the graph G , but no edges.
- 2: choose the edge e with lowest weight from graph G .
- 3: check if both vertices from e are within the same set in the tree T .
⇒ If it is not present, add the edge e to the tree T and replace

the two sets that this edge connects

4: Delete the edge e from the graph G and repeat step 2 and 3 until there is no more edge to add or until the spanning tree T contains $(n-1)$ vertices

5: Exit.

Algorithm $\Rightarrow O(E \lg V)$

MST-Kruskals (G, w)

$A \leftarrow \emptyset$

for each vertex $v \in V(G)$

do makeset (v)

Sort the edges of E in non decreasing order by weight w .

for each edge $(u, v) \in E$, taken in non decreasing order by weight.

do if find-set $(u) \neq$ find-set (v)

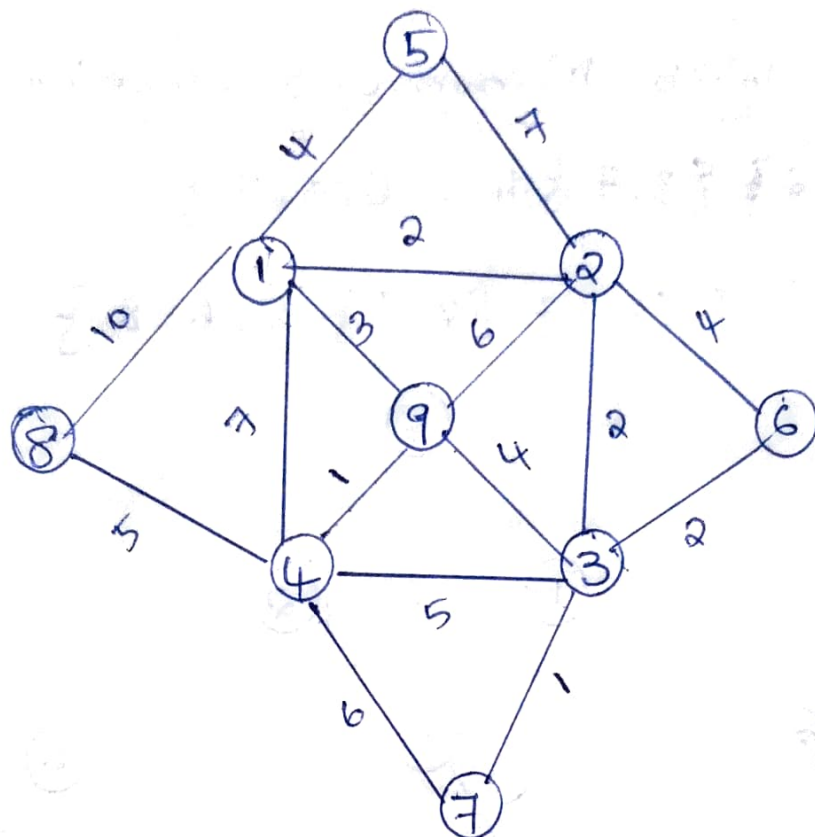
then $A \leftarrow A \cup \{(u, v)\}$

Union (u, v)

return A .

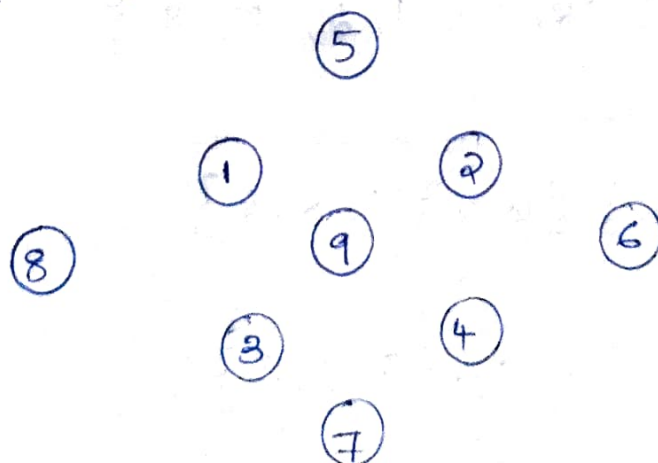
Example

Find the minimum spanning tree using Kruskal's algorithm for the graph given below



Solution

Add all vertices into the MST, ~~then add all edges into the MST~~
 $\{\{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\} \{8\} \{9\}\}$



⇒ Edge with minimum weight is selected.

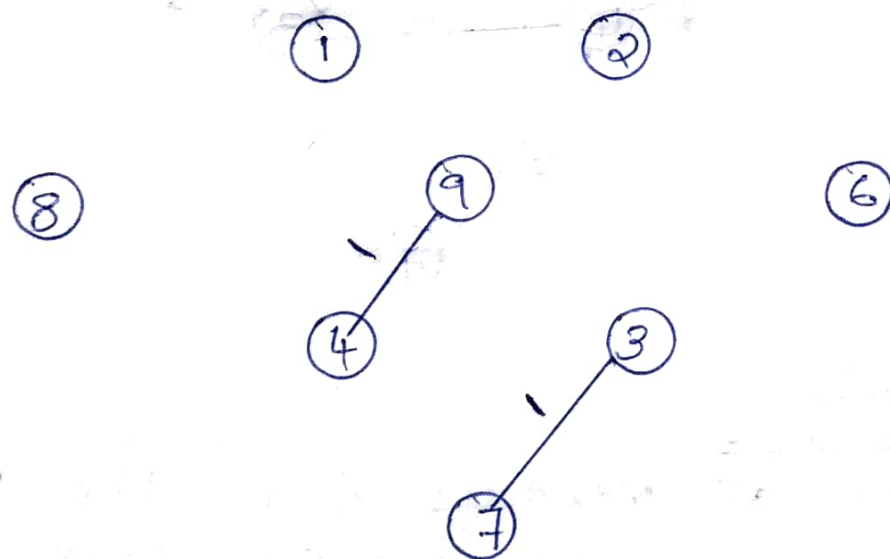
⇒ If it is not in same set, add it into same set.

Edge with Minimum Weight (7,3), (4,9)

$\{\{1\} \{2\} \{3,7\} 4,5,6,8,9\}$

$\{\{1\} \{2\} \{3,7\} \{4,9\} 5,6,8\}$

(5)



Minimum Weight edge (3,6), (3,2), (1,2)

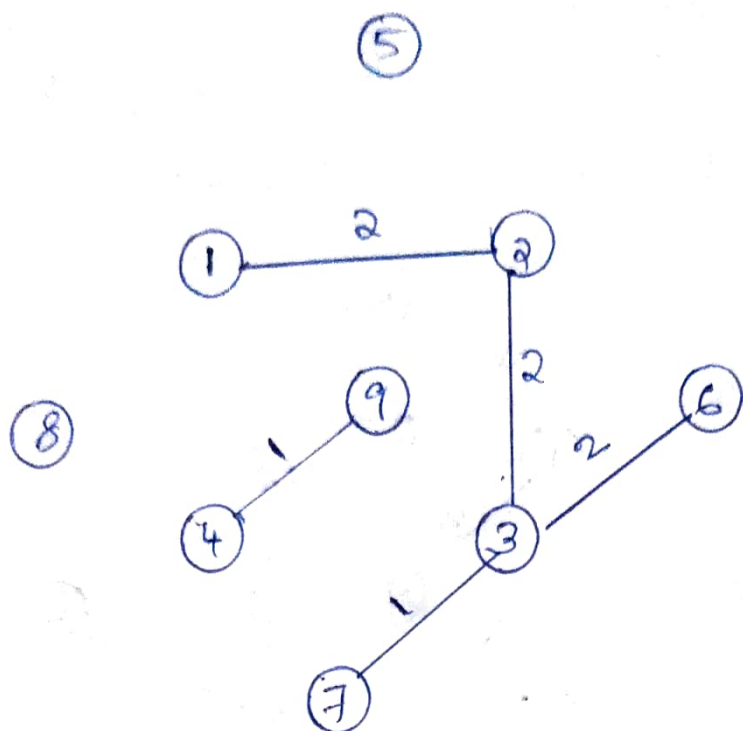
$\{\{3,7,6\} \{4,9\} 1,2,5,8\}$

⇒ (3,2)

$\{\{3,2,7,6\} \{4,9\} 1,5,8\}$

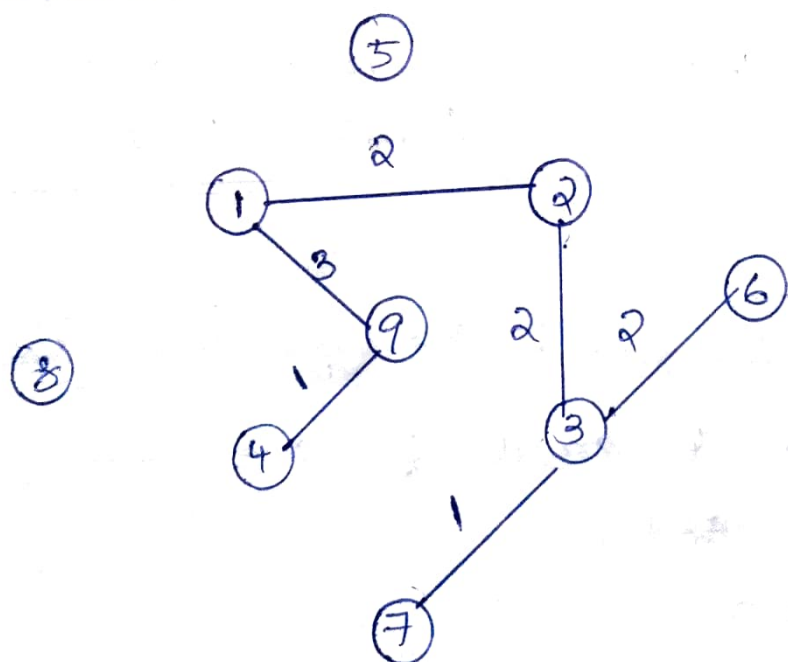
⇒ (1,2)

$\{\{1,2,3,6,7\} \{4,9\} 5,8\}$



Edge with minimum weight (1, 9)

$\{1, 2, 3, 6, 7, 4, 9\} \cup \{5, 8\}$



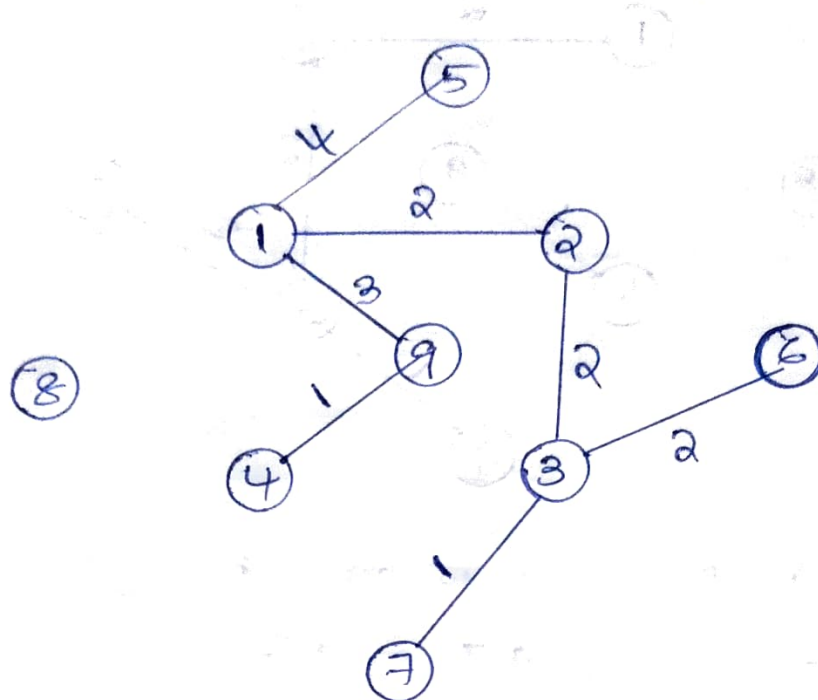
Edge with minimum weight (1, 5) (2, 6)

$\Rightarrow (1, 5)$

$\{1, 2, 3, 4, 5, 6, 7, 9\} \cup \{8\}$

$\Rightarrow (2, 6)$

$(2, 6)$ can't be joined because
2 and 6 are already in some set.



Edge with minimum weight $\Rightarrow$
 $(4, 3)$ $(4, 8)$

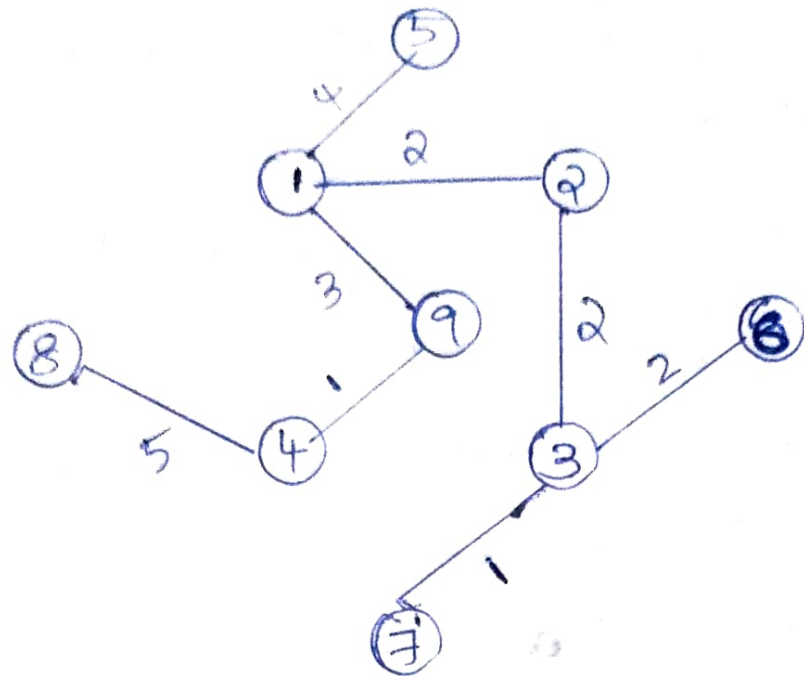
$\Rightarrow (4, 3)$

$(4, 3)$ can't be joined because
vertex 4 and 3 are already in
some set.

$\Rightarrow (4, 8)$

$\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Exit



This is the ~~minimum~~ spanning tree generated using Kruskal's algorithm.

Complexity of Kruskal's Algorithm.

⇒ To sort the edges in increasing order $\Rightarrow O(E \lg E)$

⇒ FindSet and Union operations take $O(E)$.

⇒ Along with the $|V|$ MakeSet operations these take total of $O(V + E)$ or $O(V)$ time.

⇒ Since $|E| \geq |V| - 1$ and $\alpha(|V|) = O(\log V) = O(\lg V)$

Running time is $O(E \lg V)$

PRIM'S ALGORITHM

This algorithm was discovered by Vojtech Jarník in 1936 and later rediscovered by Robert Prim.

Prim's algorithm constructs the minimum spanning tree edge by edge.

Prim's algorithm begins with a tree T , that contains a single vertex.

Prim's algorithm works as follows :-

1: Initially all nodes are made temporary

2: A node is arbitrarily selected as permanent.

3: Distance from permanent node to all other nodes are marked in the table.

4: A node with shorter distance is selected as permanent.

5: Repeat steps 3 and 4 until all nodes are made permanent.

Algorithm $\Rightarrow O(E + V \lg V)$

MST-prim's (G, w, r)

for each $u \in V(G)$

key $[u] \leftarrow \infty$

$\pi[u] \leftarrow \text{Nil}$

key $[r] \leftarrow 0$

$Q \leftarrow V[G]$

While $Q \neq \emptyset$

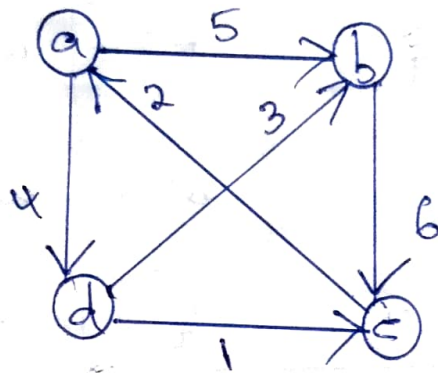
$u \leftarrow \text{Extract-Min}(Q)$

for each $v \in \text{Adj}[u]$

if $v \in Q$ and $w(u, v) < \text{key}[v]$
 then, $\pi[v] \leftarrow u$
 $\text{key}[v] \leftarrow w(u, v)$

Example

Find minimum spanning tree T ,
 for the graph given using prims
 algorithm.



Solution

$\Rightarrow$ Initially all nodes are made
 temporary

Node	Status	Key	π
a	T	∞	Nil
b	T	∞	Nil
c	T	∞	Nil
d	T	∞	Nil

⇒ A node is arbitrarily selected as permanent

⇒ Then distance from permanent node to all other nodes are marked in the table.

Node	Status	key	π
a	P	0	0
b	T	5	a
c	T	∞	∞
d	T	4	a

⇒ A node with shorter distance is selected as permanent.

⇒ Repeat this until all nodes are permanent.

Node	Status	key	π
a	P	0	0
b	T	5	a
c	T	∞	∞
d	P	4	a

a	P	0	0
b	T	3	d
c	T	1	d
d	P	4	a

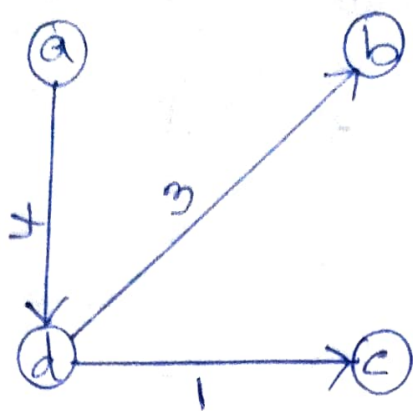
Make c Permanent

a	P	0	0
b	T	3	d
c	P	1	d
d	P	4	a

Make b permanent.

a	P	0	0
b	P	3	d
c	P	1	d
d	P	4	a

Construct Minimum spanning tree from this table.



⇒ In this example we didn't get the optimal solution.

⇒ Solution is near optimal.

This is because greedy algorithm may not give optimal solution always.

	Initial	a	d	c	b
a	∞	0	0	0	0
b	∞	5	3	3	3
c	∞	∞	1	1	1
d	∞	4	4	4	4

Using
Single
Table

Complexity of Prim's Algorithm.

⇒ steps 1-5 takes $O(V)$ time.

⇒ Therefore total time for all calls to Extract-min is $O(V \lg V)$.

⇒ steps 8-11 executes in $O(E)$ time.

⇒ step 11 altogether takes $O(\lg V)$

⇒ Therefore total time is

$$O(V \lg V + E \lg V) = O(E \lg V)$$

⇒ This can be improved by using Fibonacci heap into

$$\Rightarrow O(E + V \lg V).$$

Dijkstra's Algorithm.

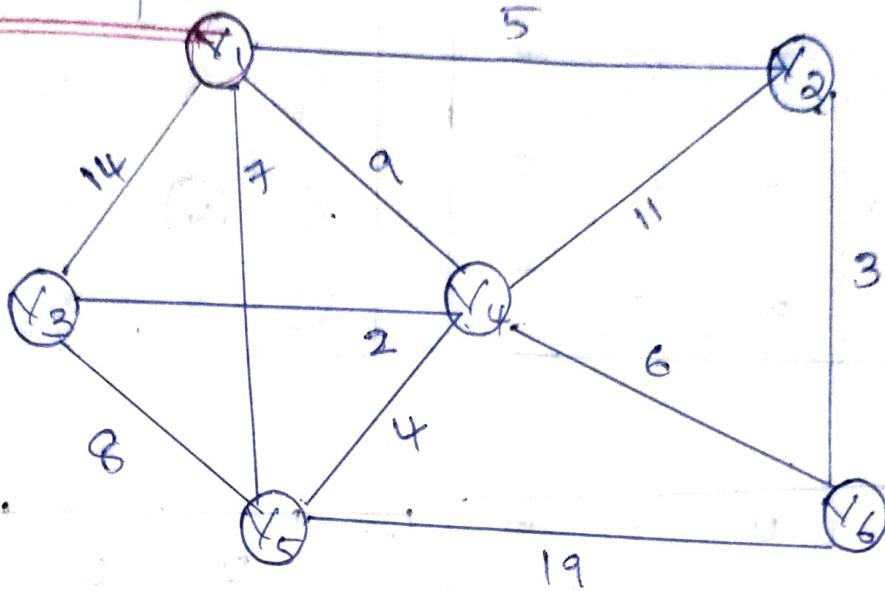
Dijkstra's algorithm solves the single source shortest path problem on a weighted, directed graph $G = (V, E)$ for the case in which all edge weights are non negative.

Procedure

- ① Initially make source node permanent and make it as the current working node. All other nodes are made to temporary.
- ② Examine all the temporary neighbours of the current working node and after checking the condition of minimum weight relabel the required node.
- ③ From all the temporary nodes find out the node which has minimum value of distance and make this node as our current working node.

- ④ Repeat steps ② and ③ until all the nodes are made permanent.
- ⑤ Stop.

Example



Find the shortest path from V_1 to V_6 .

①

Node	Distance	Predecessor	Status
V_1	0	0	P
V_2	∞	—	T
V_3	∞	—	T
V_4	∞	—	T
V_5	∞	—	T
V_6	∞	—	T

②

V_1	0	0	P
V_2	⑤	V_1	T
V_3	14	V_1	T
V_4	9	V_1	T
V_5	7	V_1	T
V_6	8	—	T

③

V_1	0	0	P
V_2	5	V_1	P
V_3	14	V_1	T
V_4	9	V_1	T
V_5	⑦	V_1	T
V_6	8	V_2	T

④

V_1	0	0	P
V_2	5	V_1	P
V_3	14	V_1	T
V_4	9	V_1	T
V_5	7	V_1	P
V_6	8	V_2	T

⑤

V_1	0	0	P
V_2	5	V_1	P
V_3	14	V_1	T
V_4	⑨	V_1	T
V_5	7	V_1	P
V_6	8	V_2	P

⑥

V_1	0	0	P
V_2	5	V_1	P
V_3	⑪	V_4	T
V_4	9	V_1	P
V_5	7	V_1	P
V_6	8	V_2	P

⑦

V_1	0	0	P
V_2	5	V_1	P
V_3	11	V_4	P
V_4	9	V_1	P
V_5	7	V_1	P
V_6	8	V_2	P

Shortest Paths

$$V_1 \longrightarrow V_2 \Rightarrow 5$$

$$V_1 \longrightarrow V_4 \longrightarrow V_3 \Rightarrow 11$$

$$V_1 \longrightarrow V_4 \Rightarrow 9$$

$$V_1 \longrightarrow V_5 \Rightarrow 7$$

$$V_1 \longrightarrow V_2 \longrightarrow V_6 \Rightarrow 8$$

Dijkstra's Algorithm

Dijkstra(G, w, v)

for each vertex $u \in V(G)$.

Key[u] $\leftarrow \infty$

$\pi[u] \leftarrow \text{Nil}$

Key[v] = 0

$Q \leftarrow V(G)$

while $Q \neq \emptyset$

$u \leftarrow \text{Extract-Min}(Q)$

remove u from Q .

for each $v \in \text{Adj}(u)$

alt = Key[u] + distance b/w
 (u, v)

if alt < Key[v]

~~alt~~ Key[v] = alt

$\pi[v] = u$.

Complexity Analysis of DIJKSTRA'S ALGORITHM.

First Can

⇒ Dijkstra's algo works by performing 3 priority queue operations

- ① Insert
- ② Extract-Min, and
- ③ Decrease Key.

⇒ Algorithm calls both Insert and Extract-Min once per vertex and Decrease key at most $|E|$ times overall.

⇒ Each Insert and Decrease key takes $O(1)$ time, and each Extract-min operation takes $O(V)$ time.

Total time ⇒ $O(V^2 + E) = O(V^2)$

Second case

If the graph is sufficiently sparse we can improve the algorithm by implementing min-priority queue with binary min heap.

⇒ Each Extract-min operation takes $O(\lg V)$ time.

⇒ Time to build binary min heap is $O(V)$.

⇒ Each decrease key operation takes $O(\lg V)$ time and there are at most $|E|$ such operations.

⇒ Therefore the total running time is $O((V+E)\lg V)$

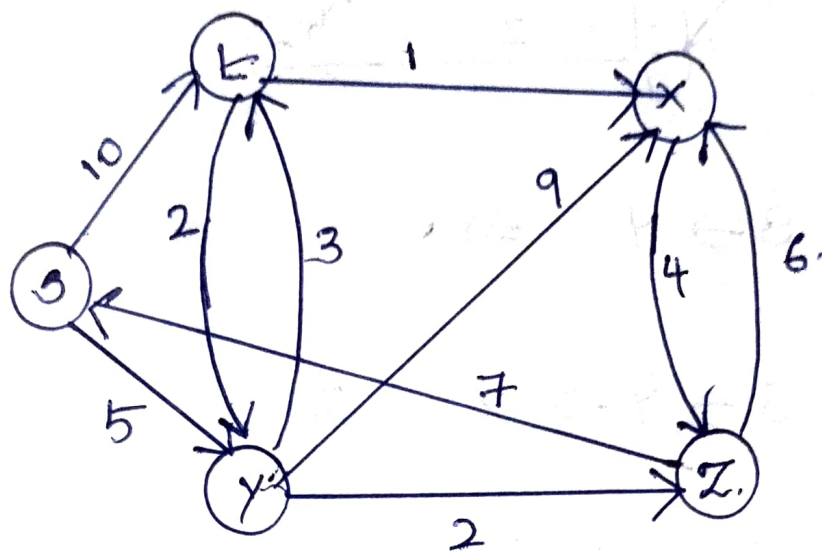
⇒ $O(E \lg V)$ if all vertices

are reachable from the source.

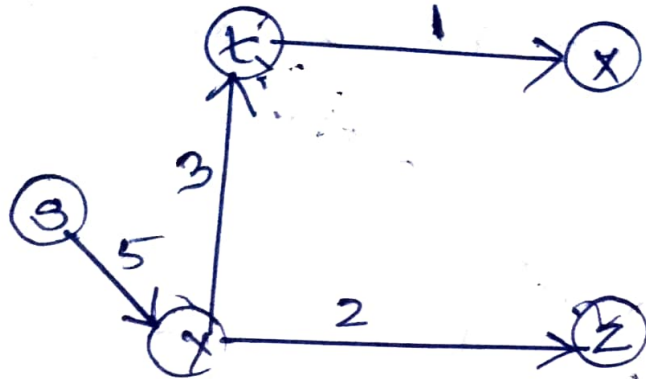
Third case

⇒ We can achieve a running time of $O(V \lg V + E)$ by implementing min-priority queue with fibonacci heap..

Q. Find out the shortest path to the graph shown below using Dijkstra's Algorithm.



	S	y	z	t	x
S	0	0	0	0	0
t	10	8	8	8	8
x	-	14	13	9	9
y*	5	5	5	5	5
z*	-	7	7	7	7



$$s \rightarrow y \Rightarrow 5$$

$$s \rightarrow y \rightarrow z \Rightarrow 7$$

$$s \rightarrow y \rightarrow t \Rightarrow 8$$

$$s \rightarrow y \rightarrow t \rightarrow x \Rightarrow 9$$

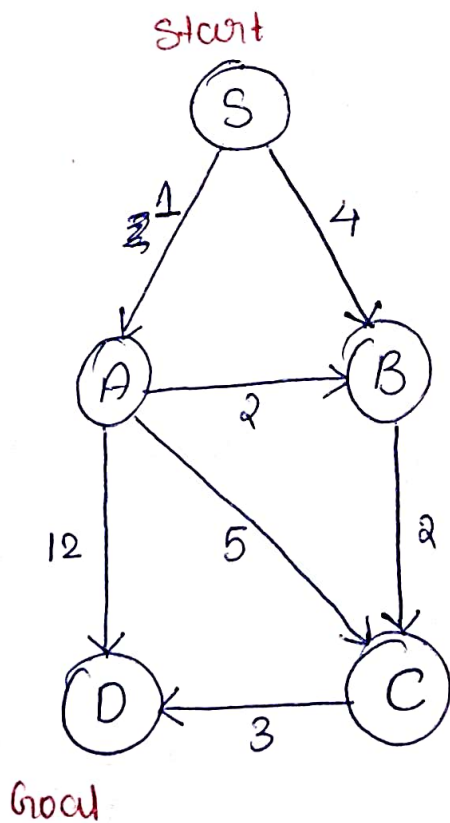
A* Algorithm

$$f(n) = g(n) + h(n)$$

$f(n)$ = estimated total path to reach goal

$g(n)$ = cost to reach the node

$h(n)$ = heuristic value (The value will be given in the question)



Given:-

heuristic value

S	7
A	6
B	2
C	1
D	0

Take node S

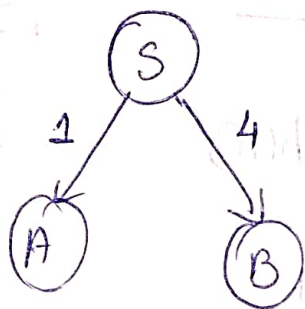
calculate $f(n)$

$$f(n) = g(n) + h(n)$$

$$= 0 + 7 = \underline{\underline{7}}$$

cost from S to S
is zero

→ from S we can either go to A or to B



$S \rightarrow A$

$S \rightarrow B$

$$f(n) = g(n) + h(n)$$

$$= 1 + 6$$

heuristic value
value of A

$$= \underline{\underline{7}}$$

$$f(n) = g(n) + h(n)$$

$$= 4 + 2$$

$$= \underline{\underline{6}}$$

~~$S \rightarrow A$~~

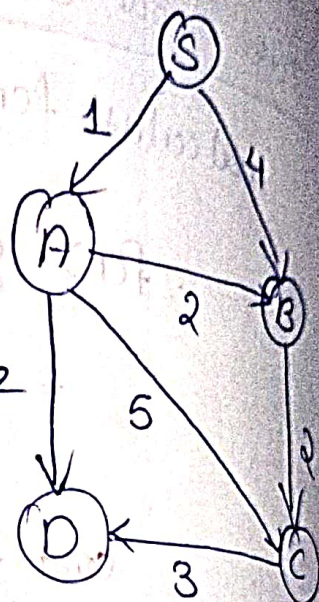
~~$S \rightarrow B$ (least value)~~

$$\underline{S \rightarrow B \rightarrow C}$$

$$f(n) = g(n) + h(n)$$

$$= 6 + 1 \quad \text{value of } C$$

$$= \underline{\underline{7}}$$



$$\underline{S \rightarrow A \rightarrow B}$$

$$f(n) = g(n) + h(n)$$

$$= 3 + 2$$

$$= \underline{\underline{5}}$$

value of B

$$\underline{S \rightarrow A \rightarrow C}$$

$$f(n) = g(n) + h(n)$$

$$6 + 1$$

$$= \underline{\underline{7}}$$

$$\underline{S \rightarrow A \rightarrow D}$$

$$f(n) = g(n) + h(n)$$

$$13 + 0$$

$$= \underline{\underline{13}}$$

value of D

$$(n) + \text{comp} = (n)$$

$$\left\{ \begin{array}{l} S \rightarrow A (7) \\ S \rightarrow B (6) \text{ (least)} \end{array} \right.$$

$$\left\{ \begin{array}{l} S \rightarrow B \rightarrow C (7) \\ S \rightarrow A \rightarrow B (5) \text{ (least)} \\ S \rightarrow A \rightarrow C (7) \\ S \rightarrow A \rightarrow D (13) \end{array} \right.$$

$S \rightarrow A \rightarrow B \rightarrow C$

$$f(n) = g(n) + h(n)$$

$$5 = 5 + 1$$

$$= 6 \checkmark$$

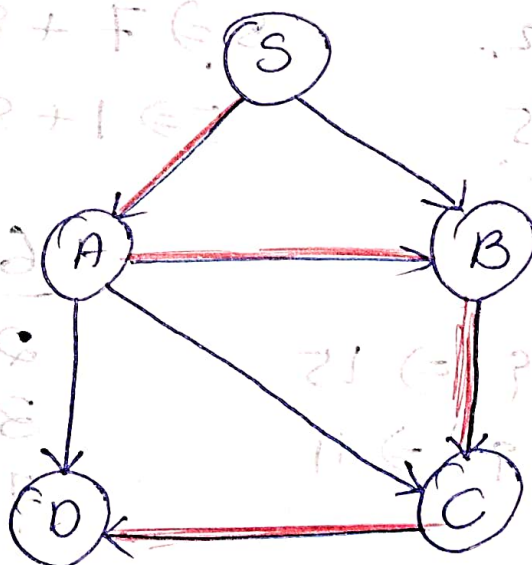
$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$

(least path.)

$$f(n) = g(n) + h(n)$$

$$= 8 + 0$$

$$= 8$$



D* Algorithm

The D* (Dynamic A*) algorithm is a pathfinding and graph search algorithm designed to efficiently find the shortest path from a start point to a goal, particularly in unknown, partially known, or changing environments. It improves on the classical A* algorithm by updating paths dynamically when new information becomes available (e.g., obstacles discovered).

Traditional algorithms like A* work well in static environments. But in real-world scenarios, such as robot navigation, the environment might change:

- New obstacles appear
- Terrain becomes non-traversable
- Sensors update the map

Instead of recalculating from scratch, D* reuses previous computations and efficiently updates only the necessary parts.

How It Works

D* is essentially a reverse A* algorithm:

- Instead of starting from the start node and searching toward the goal, D* searches backward from the goal to the start.
- This allows efficient replanning when the robot discovers changes in the environment.

When the robot detects a change (like a new obstacle):

- D* modifies costs in the affected area
- It updates the shortest path without recomputing the entire search
- It continues to use previously computed data wherever valid

Types of D*

There are several versions:

1. Original D* (1994)

- Based on A*
- Designed for real-time replanning

2. Focused D* (FD*)

- A more efficient variant with improved performance by reducing overhead in updates

3. D* Lite (2002)

- A simplified, elegant version
- Much easier to implement
- Popular in practical applications

Applications

- Autonomous robots (e.g., Mars rovers, drones)
- Real-time strategy games
- Path planning in GPS navigation systems
- Search and rescue missions

Pros and Cons

Pros

- Efficient in dynamic settings
- Avoids full recomputation
- Great for robotics

Cons

- More complex than A*
- Requires careful implementation
- Overhead in memory and data tracking

Obstacle Avoidance

1. **Purpose:** Obstacle avoidance is about **robots changing their path to avoid hitting things** detected by their sensors while moving towards a goal.
2. **Basic Principle:** The robot decides its new path based on its current sensor readings and where it wants to go.
3. **Simple Approach:** Basic methods like the **Bug algorithm** use only the most recent sensor data and a rough idea of the goal's direction.
4. **Advanced Techniques:** More sophisticated algorithms consider things like **past sensor readings**, how the robot moves (**kinematics**), and even its speed and direction (**dynamics**) for smoother and more efficient navigation.

The Bug algorithm

- The Bug algorithm is the **simplest obstacle avoidance algorithm**
- The Bug algorithm is like when you're walking and you see something blocking your path, like a wall. Instead of just walking straight into it, you follow along the wall until you find a way around it. It's the same idea for a robot – **it senses obstacles, follows their outline**, and goes around them to reach its destination.
- **Bug1** and **Bug2** are classifications of bug algorithms, which are used in robotics for pathfinding. They have slightly different approaches to navigating around obstacles, but they share the same general idea of ensuring that the robot reaches its goal while avoiding obstacles in its path.

Bug1

- Imagine **Bug1** as a robot trying to reach a goal while avoiding obstacles. Instead of heading straight to the goal, it circles around obstacles first, like walking around a parked car to reach the other side of the street. Once it's done circling, it picks the closest point to the goal and moves towards it. While this method guarantees it eventually reaches the goal if it's possible, it's not the quickest way because of all the circling around obstacles.

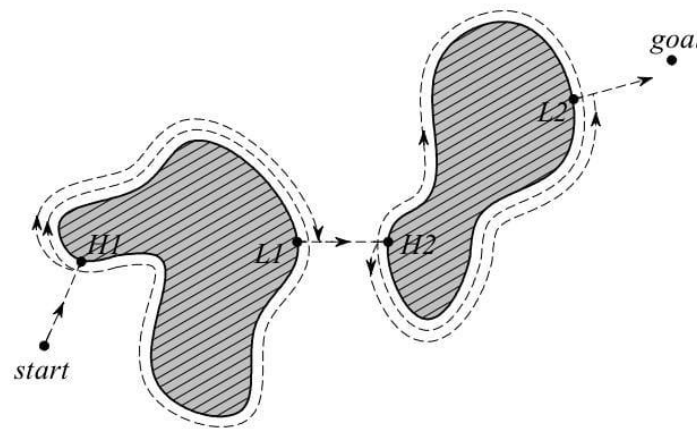


Figure 6.7

Bug1 algorithm with H1, H2, hit points, and L1, L2, leave points [102].

Bug2

- Bug2 is another type of algorithm used by robots to navigate around obstacles and reach a goal. Unlike Bug1, Bug2 starts following the contour of the obstacle but will immediately head towards the goal if it can move directly to it without any obstacles in the way. This means **Bug2 tends to have shorter travel distances compared to Bug1** because it doesn't always fully circle obstacles. However, there are still situations where Bug2 might not take the most optimal path and could be inefficient in reaching the goal.

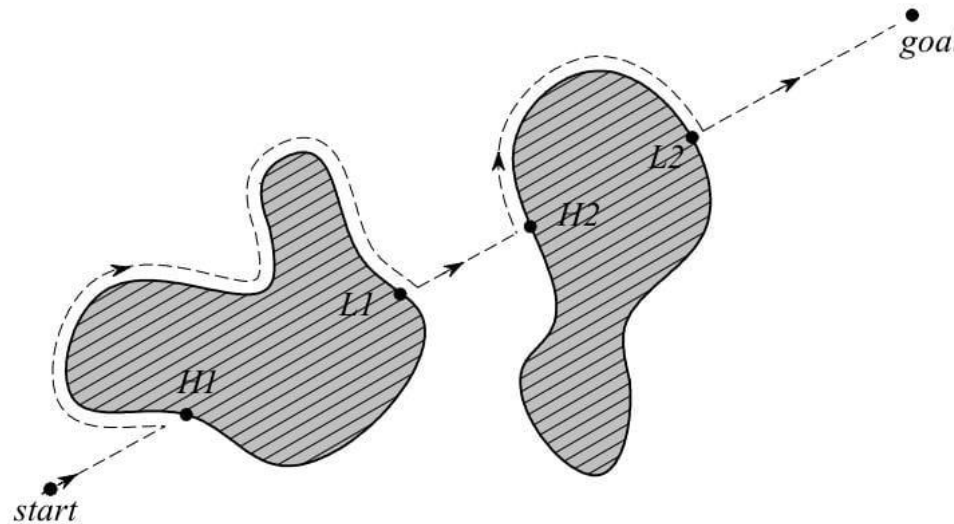


Figure 6.8

Bug2 algorithm with H1, H2, hit points, and L1, L2, leave points [102].

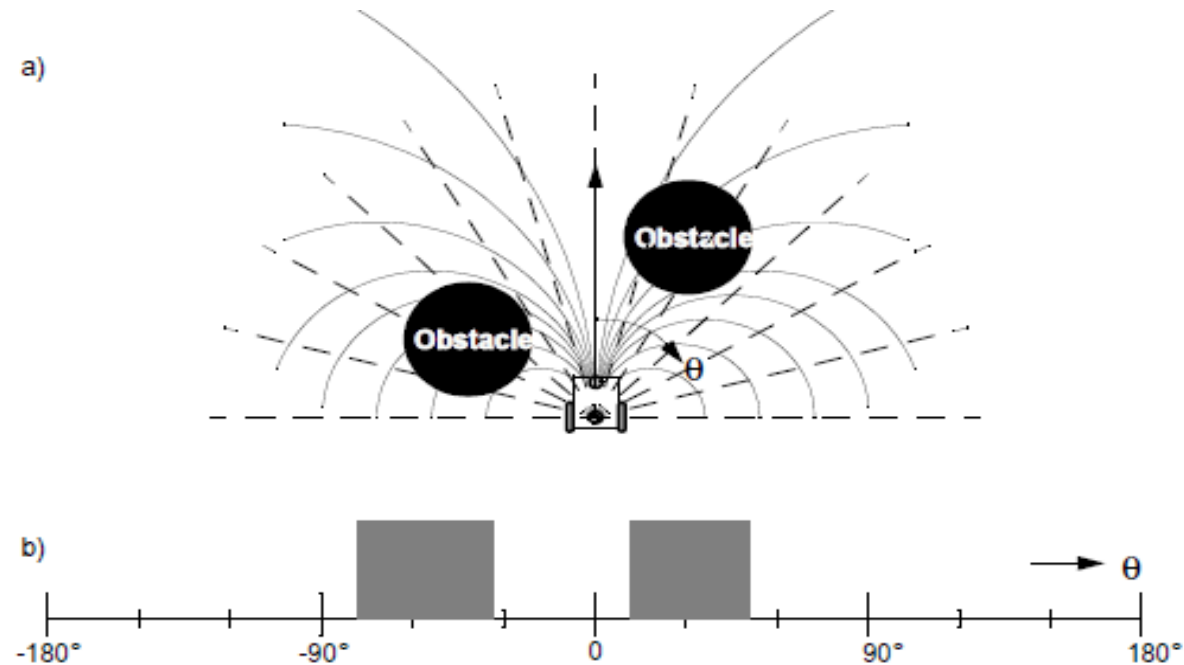
Tangent Bug algorithm

- The Tangent Bug algorithm is an **extension of the Bug algorithm** used by robots to find their way around obstacles to reach a goal. What makes Tangent Bug different is that it has sensors to detect obstacles and **creates a sort of map called the local tangent graph (LTG)** to understand the environment better. With this extra information, the robot can find faster routes to its destination by going around obstacles and heading straight towards the goal sooner. In simpler environments, Tangent Bug can even find paths that are close to the best possible ones.

VFH(vector field histogram)

- Bug-type algorithms sometimes have a problem because they only consider the robot's most recent sensor readings, which might not always give enough information for the robot to avoid obstacles effectively. **VFH techniques** solve this by creating a mini-map of the area around the robot. This map is like a tiny grid showing where obstacles are based on recent sensor readings. Then, VFH uses this map to create a chart showing the likelihood of obstacles in different directions. This helps the robot better navigate around obstacles, even when its sensors don't provide perfect information.
- Imagine you're standing at the center of a circle, and you're surrounded by sensors that can detect obstacles at different angles around you. The polar histogram created by VFH is like a chart that shows these angles.

- The VFH techniques overcome this limitation by creating a local map of the environment around the robot.
- For obstacle avoidance, VFH generates a polar histogram as shown in figure.
- The x-axis represents the angle at which the obstacle was found and
- the y-axis represents the probability that there really is an obstacle in that direction



Dynamic window approaches

- Another technique for obstacle avoidance.
- A simple but very effective dynamic model.
- **The local dynamic window approach.**
- **The global dynamic window approach.**

The local dynamic window approach

- Imagine you're driving a car. Before you make a turn, you quickly check your surroundings to make sure it's safe. You consider your car's capabilities, like how fast you can go and how sharply you can turn. You also think about the space available on the road to maneuver.
- Similarly, in the local dynamic window approach for robots, the robot evaluates its surroundings to find the best speed and direction to move. But instead of roads, it considers its velocity space, which includes all the possible combinations of speed and angular (turning) velocity.

- 1. Evaluate Surroundings:** The robot looks at what's around it using sensors like cameras or lidars. It identifies obstacles and determines where it wants to go.
- 2. Consider Kinematics:** The robot knows its own capabilities in terms of speed and turning ability. It looks at its velocity space, which is like a map showing all the different speeds and turn rates it can achieve.
- 3. Search for Best Motion:** Within this velocity space, the robot searches for the best combination of speed and turning that allows it to move toward its goal while avoiding obstacles. It's like finding the best route on a map.
- 4. Move in Circular Arcs:** Once it finds the best combination, the robot moves in curved paths called circular arcs. These arcs represent its movement for a short period, like a snapshot in time.
- 5. Update Continuously:** The robot keeps updating this process as it moves, constantly reassessing its surroundings and adjusting its velocity to navigate safely and efficiently.

The global dynamic window approach.

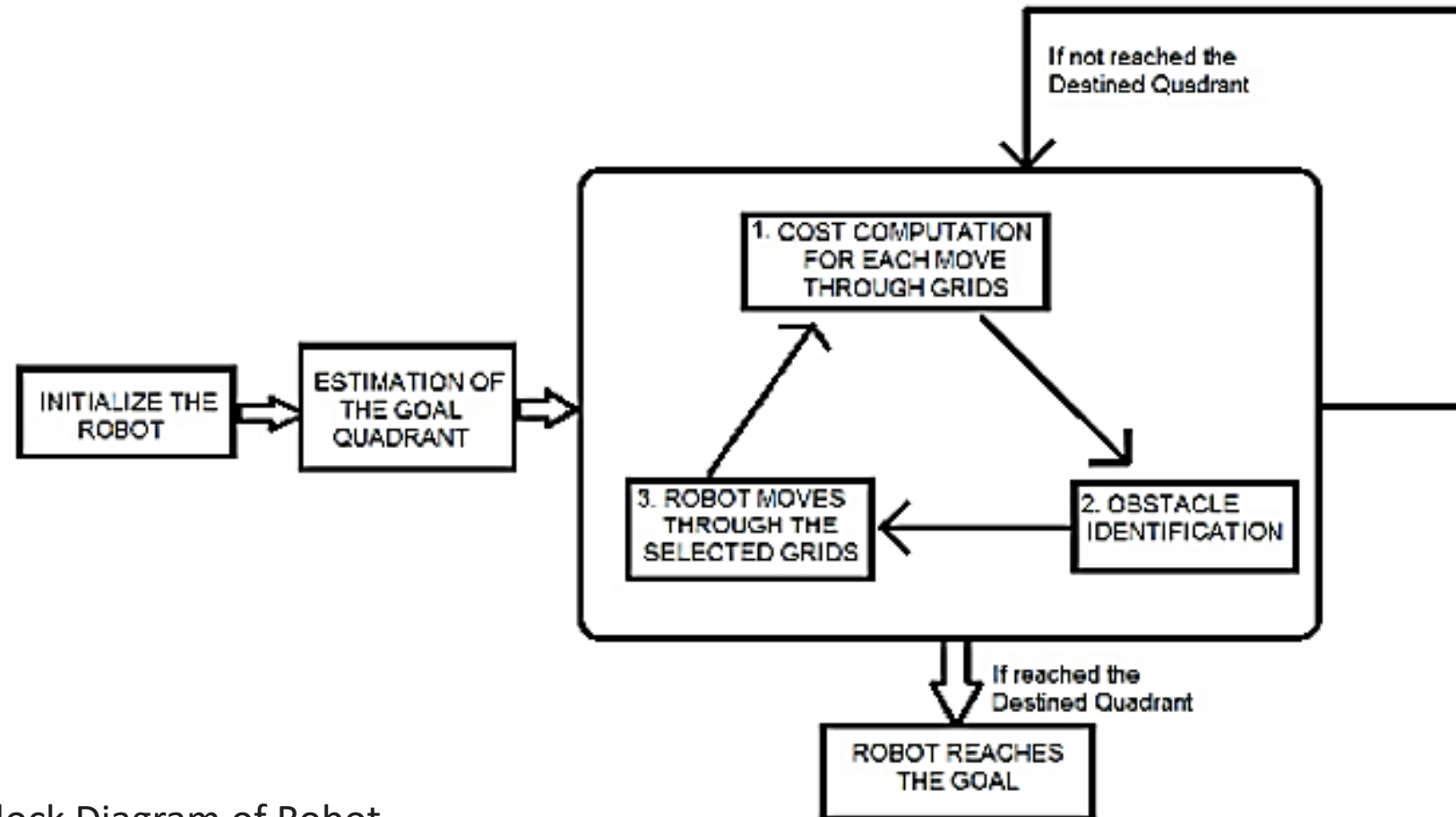
- The "global window approach" in obstacle avoidance refers to a strategy where a robot considers a larger area of its environment when planning its path to avoid obstacles.
- Instead of only focusing on obstacles nearby, like in local obstacle avoidance methods, the robot looks at a wider area, often covering its entire planned path or a significant portion of its environment.
- By considering obstacles and potential paths over a larger distance, the robot can make strategic decisions about its route. It's like looking ahead on a road trip and planning your route based on what you see on a map.
- The global window approach allows the robot to anticipate obstacles further ahead and plan its path accordingly. This helps the robot avoid getting stuck or making sudden changes in direction when it encounters obstacles.
- By planning ahead and considering the entire environment, the robot can navigate more efficiently, choosing the best path to reach its destination while avoiding obstacles along the way.

Navigation

- Navigation is the process of determining and controlling the movement of an object, such as a vehicle or robot, from one location to another.
- It involves planning routes, avoiding obstacles, and executing movements to reach desired destinations efficiently and safely.

Navigation architecture

- Navigation architecture in robotics refers to the framework or system design used for guiding a robot through its environment.
- It encompasses various components and algorithms that enable the robot to perceive its surroundings.
- plan a path to navigate from one point to another, and execute this path while avoiding obstacles and reaching its destination safely.



The Architectural Block Diagram of Robot Navigation is given. After initializing the robot, it starts moving the target location and tracks the path and the robot will make movements according to the Enhanced algorithms through the destined path and reaches the target

Key components of navigation architecture in robotics typically include:

- 1. Perception:** This involves sensors such as cameras, LiDAR (Light Detection and Ranging), radar, sonar, and others, which gather data about the robot's environment.
- 2. Localization:** Localization algorithms determine the robot's position within its environment. This often involves techniques such as Simultaneous Localization and Mapping (SLAM), which allow the robot to build a map of its surroundings while simultaneously determining its own position within that map.
- 3. Path Planning:** Path planning algorithms calculate a safe and efficient path for the robot to follow from its current location to its desired destination.
- 4. Obstacle Avoidance:** Obstacle avoidance algorithms ensure that the robot can navigate around obstacles it encounters along its path.
- 5. Control:** Control systems translate the planned path into specific commands for the robot's actuators (such as motors or wheels) to execute.

- Given techniques for path planning, obstacle avoidance, localization, and perceptual interpretation,
- how do we combine all of these into one complete robot system for a real-world application?
- One way to proceed would be to custom-design an application-specific, monolithic software system that implements everything for a specific purpose.
- for any sophisticated and long-term mobile robot system, the issue of mobility architecture should be addressed in a principled manner.
- The study of *navigation architectures* is the study of principled **designs for the software modules** that constitute a mobile robot navigation system.

Modularity for code reuse and sharing

- In robotics, modularity means breaking down the robot's software into smaller, independent parts or modules. Just like building blocks, these modules can be combined and reused in different robot projects.
- Why is this important? Because robots often encounter different situations and environments. For example, they might use different sensors or move in different ways. Modularity helps us adapt the robot's abilities without causing problems elsewhere.
- For instance, let's say we want to upgrade a robot's sensors from ultrasonic to laser. With modularity, we can easily swap out the sensor module without needing to change the whole system. This way, the robot can still navigate and avoid obstacles smoothly.
- In simple terms, modularity in robotics is like having interchangeable parts that allow robots to adapt to new challenges without starting from scratch.

Control localization

- localize all software related to the robot
- When we talk about controlling a robot, especially when it's moving around, like in navigation, it's important to organize how we handle different tasks, like avoiding obstacles or deciding where to go. This is where localization comes in.
- **Localization means assigning specific tasks or functions to different parts of the robot's brain**, so to speak. For example, we might have one part of the robot's "brain" dedicated to avoiding obstacles, while another part focuses on figuring out where the robot is in its environment. By doing this, we can test each part separately and make sure they work well together.
- **Imagine you're driving a car. You have different tasks to do: steering, accelerating, braking, checking mirrors, etc. Each task requires focus, but they all come together to help you drive safely. Similarly, in a robot, we divide tasks like this so that we can manage them effectively.**
- **By organizing robot control in this way, we can test and improve each part independently**, making the robot smarter and more reliable overall. It's like having a well-organized team where each member knows their role and works together smoothly to accomplish the goal of safely navigating the robot through its environment.

Techniques for decomposition

- Decomposition, in the context of robotics or any complex system, refers to the process of breaking down a problem, task, or system into smaller, more manageable parts or components. These smaller parts are often easier to understand, develop, test, and maintain individually, which can lead to more efficient and effective overall system design and operation.
- **Temporal decomposition**
- **Control decomposition**

Temporal decomposition

- This is like dividing a robot's tasks based on time. Some tasks need to be done quickly, in real-time, while others can take longer. For example, when a robot is avoiding an obstacle, it needs to react quickly to avoid a collision—that's a real-time task. But planning a long-term path to a destination can take more time and doesn't need immediate responses.

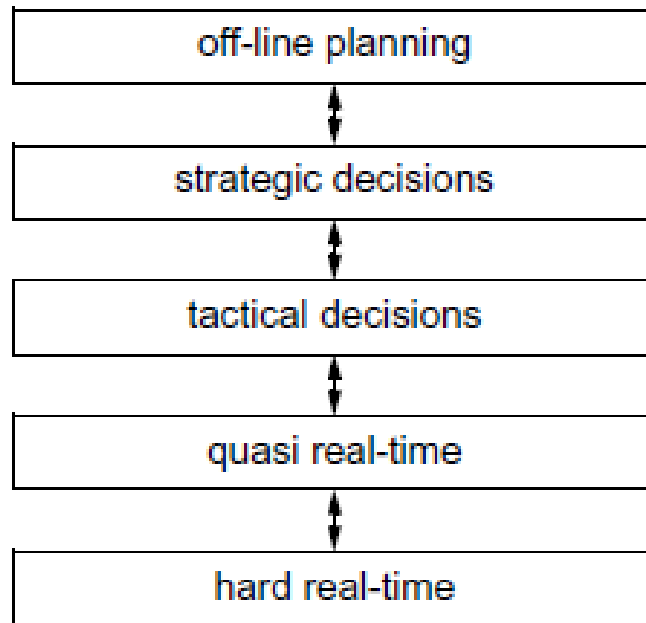


Figure 6.18
Generic temporal decomposition of a navigation architecture.

1.Offline Planning: Imagine you're going on a road trip and you plan your route on a map before you leave. This is similar to offline planning for a robot. It involves figuring out the best path or route to take to reach a destination before the robot starts moving. The robot doesn't need to make decisions in real-time during offline planning; it's like creating a blueprint for its journey.

2.Strategic Decisions: These are the big-picture decisions made before the robot starts moving. For example, deciding which route to take for the entire journey falls under strategic decisions. It's like deciding on the overall strategy or plan for reaching the destination.

3.Tactical Decisions: Once the robot starts moving, it may encounter unexpected situations, like obstacles or detours. Tactical decisions are made in response to these immediate challenges. For instance, if the robot encounters a closed road, it needs to quickly decide on an alternative route. Tactical decisions are like adjustments made on the fly to keep the robot on track towards its destination.

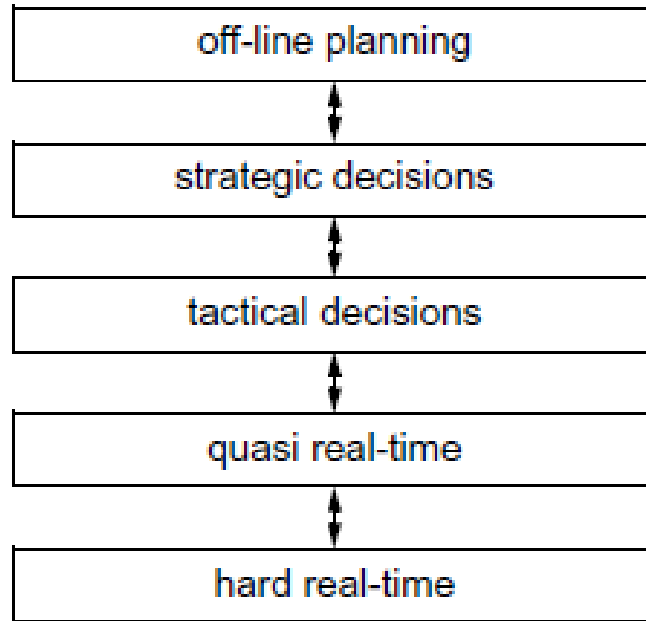


Figure 6.18

Generic temporal decomposition of a navigation architecture.

1. Quasi-real-time: This means decisions are made quickly, but not necessarily instantly. It's like having a short delay between sensing an obstacle and reacting to it. Quasi-real-time decisions are fast, but they may not happen in the split-second timing of true real-time decisions.

Imagine a self-driving car traveling down a city street. As it approaches an intersection, it uses its sensors to detect pedestrians crossing the road. The car's onboard computer processes this information quickly and determines that it needs to slow down to avoid a potential collision.

2. Hard Real-time: This refers to decisions that must be made instantly, with absolutely no delay. For example, if a robot is about to collide with an obstacle, it needs to stop immediately to avoid a crash. Hard real-time decisions are critical for safety and must happen in the blink of an eye.

Control Decomposition

- This is about breaking down how a robot's actions are controlled. Think of it like dividing up the controls of a car. You have the steering wheel to control direction, the pedals to control speed and braking, and the gear shifter to control movement forward or backward. Similarly, a robot's control system has different parts that work together to make it move and behave the way it should.

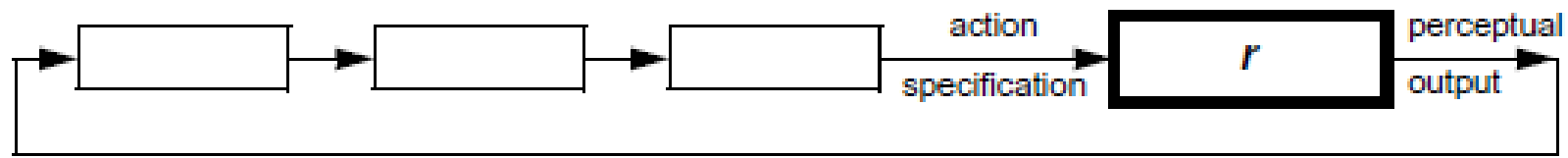


Figure 6.20
Example of a pure serial decomposition.

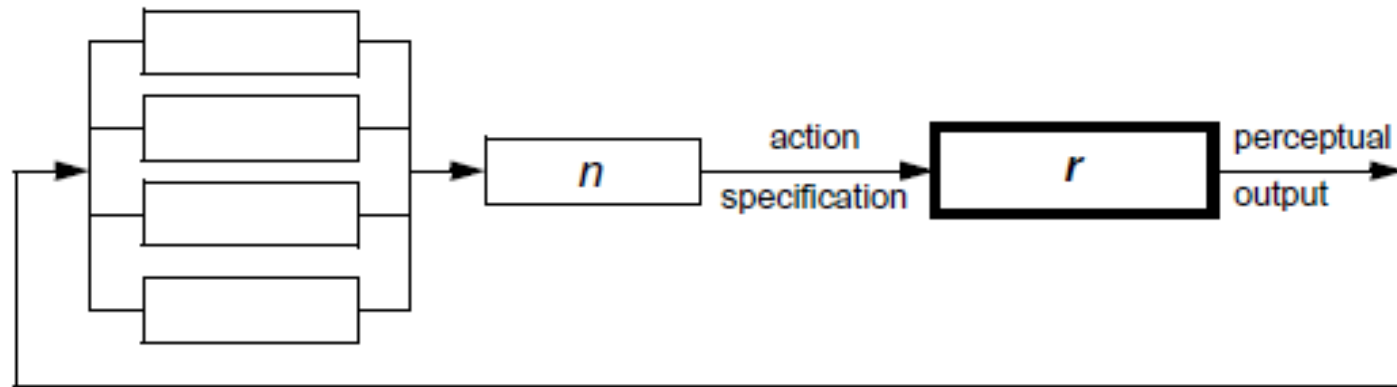


Figure 6.21
Example of a pure parallel decomposition.

1.Series Decomposition:

1. In series decomposition, tasks or components are arranged in a sequence, one after the other.
2. It's like a relay race, where each runner passes the baton to the next one in line.
3. In a series setup, the output of one task becomes the input for the next task.
4. For example, in a series of tasks for baking a cake, you might first mix the ingredients, then pour the batter into a pan, and finally bake it in the oven. Each step depends on the previous one.

2.Parallel Decomposition:

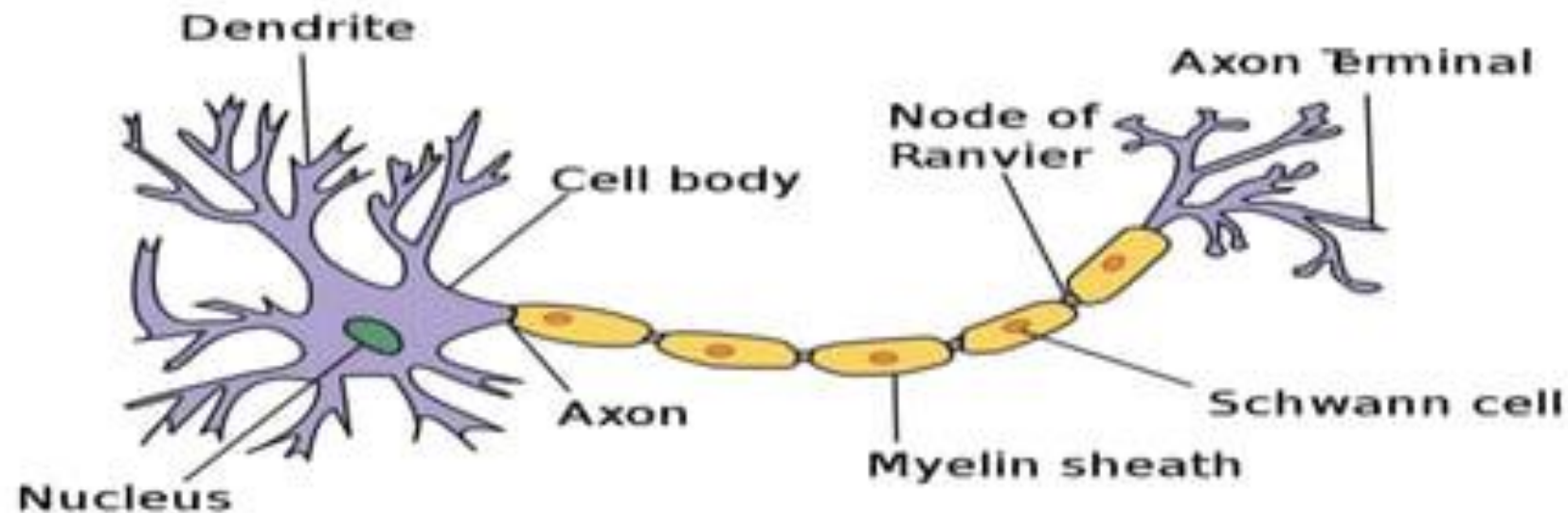
1. In parallel decomposition, tasks or components are arranged to run simultaneously or independently of each other.
2. It's like having multiple lanes on a highway, where cars can travel side by side without depending on each other.
3. In a parallel setup, tasks can be performed concurrently, and their outputs don't necessarily affect each other.
4. For example, in baking a cake, you might have one person mixing the ingredients while another person preheats the oven. These tasks can happen at the same time without one depending on the other.

Neural Networks

MODULE 5

Introduction

A neuron is a cell that carries electrical impulses. Neurons are the basic (functional & structural) units of our nervous system.



Overview of Neural Networks

- A neural network (NN) is a computational model inspired by the way biological neural systems, such as the human brain, process information. It consists of layers of neurons that work together to learn patterns from input data, such as images, and make predictions or decisions based on that data.
- When processing images, the neural network typically follows these steps:
- **Input Layer:** The raw pixel values of the image are fed into the network. For grayscale images, the input will be a 2D array of pixel values. For colored images, this might be a 3D array with each color channel (RGB) representing separate layers.
- **Hidden Layers:** The hidden layers are where the network "learns" features from the image. These layers consist of artificial neurons that apply weighted sums and activation functions to the input data.
- **Output Layer:** The final layer produces the network's predictions, such as object recognition or classification labels.

Processing the image

- When we use a **neural network** to process an image, the goal is to enable the network to **understand** and **interpret** the features of the image in a way that allows it to make decisions, such as classifying objects, detecting objects, or generating new images.
- Before we feed an image into a neural network, the image needs to be represented numerically so the network can process it. Typically, an image is represented as a matrix or tensor of pixel values. Each pixel in an image corresponds to a number representing the intensity or color value at that point. The way an image is processed in a neural network depends on whether it is in grayscale or color.

1. Preprocessing:

To prepare the image for processing, several preprocessing steps are often applied:

1. **Resizing:** Images might be resized to a fixed size
2. **Normalization:** Pixel values are often normalized to the range of $[0, 1]$ or $[-1, 1]$ to speed up convergence during training.
3. **Data Augmentation:** Techniques like flipping, rotation, scaling, and cropping are used to artificially expand the dataset and make the model more robust.

2. Passing the Image through Neural Network Layers

Once the image is preprocessed, it is fed into the neural network, where it passes through several layers. The two most common types of neural networks for image processing are **Fully Connected Networks** and **Convolutional Neural Networks (CNNs)**.

- **Fully Connected Networks (Dense Networks)**
- In the case of a **fully connected network**:
- Each pixel value from the image (flattened into a 1D array) is passed into a series of connected neurons.
- Each neuron applies a weighted sum of inputs, followed by an activation function (e.g., ReLU, Sigmoid, etc.), to process the information.
- However, fully connected networks are less efficient and less powerful for image processing compared to **Convolutional Neural Networks (CNNs)**.

- **Convolutional Neural Networks (CNNs)**

CNNs are specifically designed for image data, and they process images in a way that mimics the human visual system. CNNs leverage spatial hierarchies of features (i.e., patterns like edges, textures, and objects) by using convolutional layers.

Layers in a CNN for Image Processing:

- **Convolutional Layer (Conv Layer):**
- **Convolution** is the core operation in a CNN. It applies a small **filter (kernel)** over the input image (or the feature maps from the previous layer) to extract local patterns such as edges, corners, and textures.
- For instance, a 3x3 kernel could slide over the image and calculate the weighted sum of the pixel values it covers. The resulting map is called the **feature map**.
- Multiple filters are used to capture different features, and each filter generates its own feature map.
- **Example:** In the first convolutional layer, the network may detect simple patterns such as horizontal or vertical edges.

- **Activation Layer :**
- After each convolution, an **activation function** is applied to introduce non-linearity, allowing the network to learn more complex patterns.
- **ReLU** (Rectified Linear Unit) transforms the output of the convolution by setting all negative values to zero and leaving positive values unchanged.
- **Pooling Layer:**
- **Pooling** reduces the spatial dimensions (height and width) of the feature maps, making the network less computationally expensive and more robust to variations in the input.
- **Max pooling** is most commonly used, where for each local region, the maximum value is taken to create a smaller representation.
- **Example:** A 2x2 max pooling operation will reduce a 4x4 feature map to a 2x2 feature map, keeping only the highest values.

- **Fully Connected (Dense) Layer:**
- After several convolutional and pooling layers, the network typically includes one or more **fully connected layers**.
- These layers take the flattened feature map and connect every neuron to all neurons in the next layer.
- These layers are responsible for high-level reasoning and final classification or decision-making.
- **Output Layer:**
- The final layer outputs the prediction. For classification tasks, this is often a softmax layer that converts the output into a probability distribution over the class labels.
- For regression or other tasks, the output layer might be a single neuron with linear activation.

4. Training the Neural Network for Image Processing

- During training, the network adjusts its weights to minimize the error in predictions using the **backpropagation** algorithm:
 1. **Forward Pass:** The input image passes through the network layers, and the final output (prediction) is computed.
 2. **Loss Calculation:** The predicted output is compared to the true label (ground truth), and a loss function (e.g., cross-entropy loss for classification) is used to calculate the error.
 3. **Backpropagation:** The error is propagated back through the network to adjust the weights using optimization algorithms like **Stochastic Gradient Descent (SGD)** or **Adam**.

5. Benefits of Using CNNs for Image Processing

- **Feature Extraction:** CNNs automatically learn hierarchical features from the raw pixel data, reducing the need for manual feature engineering.
- **Spatial Invariance:** Due to pooling and local connectivity, CNNs are less sensitive to small translations and distortions in the image.
- **Parameter Sharing:** Filters in convolutional layers are shared across the image, drastically reducing the number of parameters in the network and improving efficiency.

